

DISCIPLINA ANÁLISE E PROJETOS ORIENTADA A OBJETOS



SUMÁRIO

UNIDADE 1.....	3
Introdução.....	3
Conceitos fundamentais de orientação a objeto	3
Conceitos fundamentais de análise e projetos OO.....	7
Iterativo, Incremental e Ágil	9
Considerações Finais	12
UNIDADE 2	13
Introdução.....	13
Concepção.....	14
Requisitos Evolutivos.....	16
Casos de Uso.....	18
Considerações Finais	22
UNIDADE 3	23
Introdução.....	23
Diagrama de Classes.....	23
Diagrama de Atividade.....	30
Diagrama de Máquina de Estados	31
Considerações Finais	37
UNIDADE 4	38
Introdução.....	38
Separação de Responsabilidades	38
Domain Driven Development (DDD)	41
BPMN, DMN e CMMN.....	43
Considerações Finais	48
UNIDADE 5	49
Introdução.....	49
Boas Práticas de Projetos.....	52
Considerações Finais	53
Referências.....	54

APRESENTAÇÃO

Everton Gomede

Doutorando em Engenharia Elétrica/Computação pelo Programa de Pós-graduação da Faculdade de Engenharia Elétrica e de Computação (FEEC) da Universidade Estadual de Campinas (UNICAMP). Mestre em Ciência da Computação pela Universidade Estadual de Londrina. MBA (Master of Business Administration) em Neurogestão pela Fundação Getúlio Vargas. MBA em Gerenciamento de Projetos pela FGV. Especialista em Engenharia de Software e Banco de Dados pela Universidade Estadual de Londrina. Graduado em Processamento de Dados. Sun Certified Programmer for Java 2 Platform 1.4 e ITIL® Foundation Certificate in IT Service Management. Experiência de mais de 10 anos na área de TI em arquitetura de software, engenharia de software, mapeamento de processos de negócio, gerenciamento de projetos, desenvolvimento de sistemas transacionais e de suporte à decisão. Experiência em projetos nacionais e internacionais em instituições financeiras e empresas de desenvolvimento de software. Professor de cursos de pós-graduação em universidades públicas e privadas. Membro do PMI 1580489. Publicações nacionais e internacionais. Proficiência em inglês atestada pelo TOIEC e TOEFL. Áreas de Interesse: Estrutura de Dados e Organização de Arquivos, Análise de Sistemas e Engenharia de Software, Lógica de Programação, Redes de Computadores, Linguagem de Programação, Administração de Banco de Dados, Governança de TIC (ITIL/CobIT), Data Warehouse, Data Mining, Big Data, Gerenciamento de Projetos, Gerenciamento de Processos de Negócio (BPM/BPMN™), Pesquisa Operacional, Algoritmos, Sistemas de Suporte à Decisão, Data Science e Machine Learning.

Ricardo Satin

Mestre em Engenharia de Software pela UTFPR; Especialista em Computação pela Unicesumar; Graduado em Computação pela Universidade Estadual de Maringá -UEM; MBA em Gerenciamento de Projetos pela FGV. Profissional certificado em: gerenciamento de projeto pelo P.M.I. (profissional PMP); Gerenciamento ágil de projetos (SCRUM); Gerenciamento de serviços de T.I. (ITIL). Professor de ensino superior para cursos da área de T.I. e pós-graduação para diversas áreas. Atua como consultor em gerenciamento de projetos; processos de negócio; melhoria de processos e machine learning.

UNIDADE 1

VISÃO GERAL

Objetivos de aprendizagem

- Aprender os conceitos fundamentais de orientação a objeto
- Aprender os conceitos fundamentais de análise e projetos orientado a objeto
- Aprender os modelos iterativo, incremental e ágil

Introdução

Nesta unidade vamos estudar os conceitos fundamentais de orientação a objetos e como os mesmo podem ser utilizados para análise e projeto. Vamos estudar também os detalhes de como funciona a análise e como a mesma auxilia usuários e desenvolvedores a desenvolverem software de qualidade. Adicionalmente, vamos estudar os aspectos dos projetos orientado a objeto e como isso melhora o resultado do software a ser construído.

Conceitos fundamentais de orientação a objeto

O desenvolvimento de software é extremamente amplo (LARMAN, 2007). Nesse contexto, existem diversas linguagens de programação, que obedecem a diferentes paradigmas (ENGHOLM, 2017). Um desses paradigmas é a *Orientação a Objetos* (OO), que atualmente é o mais difundido entre todos (FREEMAN, 2007). Isso acontece porque se trata de um padrão que tem evoluído muito, principalmente em questões voltadas para segurança e reaproveitamento de código, o que é muito importante no desenvolvimento de qualquer aplicação moderna de software.

A *Programação Orientada a Objetos* (POO) diz respeito a um padrão de desenvolvimento que é seguido por muitas linguagens, como C# e Java. A seguir, vamos entender as diferenças entre a POO e a Programação Estruturada, que era muito utilizada há alguns anos, principalmente com a linguagem C. Na Figura 1, abaixo, vemos uma comparação muito clara entre a programação estruturada e a programação orientada a objetos no que diz respeito aos dados. Repare que, no paradigma estruturado, temos procedimentos (ou funções) que são aplicados globalmente em nossa aplicação. No caso da orientação a objetos, temos métodos que são aplicados aos dados de cada objeto. Essencialmente, os procedimentos e métodos são iguais, sendo diferenciados apenas pelo seu escopo (WAZLAWICK, 2014).

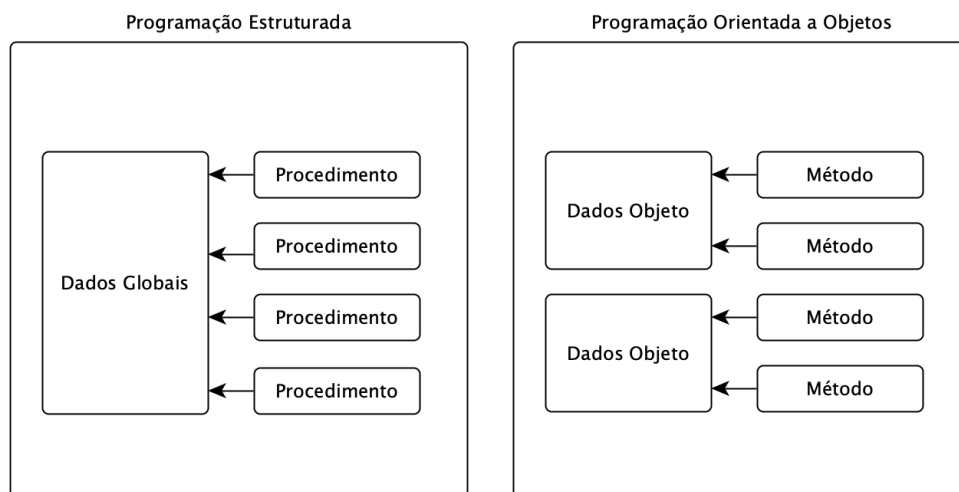


Figura 1 – Paradigma Estruturado x Orientação a Objeto

A linguagem C é a principal representante da programação estruturada. Se trata de uma linguagem considerada de baixo nível, que atualmente não é utilizada para projetos muito grandes (ou, principalmente, projetos de natureza web com a necessidade de executar em uma gama de diferentes dispositivos). A sua principal utilização, devido ao baixo nível, é em programação para sistemas embarcados ou outros em que o conhecimento do hardware se faz necessário para um bom software.

Essa colocação nos traz a um detalhe importante: a programação estruturada, quando bem feita, possui um desempenho superior ao que vemos na programação orientada a objetos (GAMMA, 2000). Isso ocorre pelo fato de ser um paradigma sequencial, em que cada linha de código é executada após a outra, sem muitos desvios, como vemos na POO. Além disso, o paradigma estruturado costuma permitir mais liberdade com o hardware, o que acaba auxiliando na questão desempenho.

Entretanto, a programação orientada a objetos traz outros pontos que acabam sendo mais interessantes no contexto de aplicações modernas. Como o desempenho das aplicações não é uma das grandes preocupações na maioria das aplicações (devido ao poder de processamento dos computadores atuais), a programação orientada a objetos se tornou muito difundida. Essa difusão se dá muito pela questão da reutilização de código e pela capacidade de representação do sistema muito mais perto do que veríamos no mundo real (ENGHOLM, 2017).

Para entendermos exatamente do que se trata a orientação a objetos, vamos entender quais são os requerimentos de uma linguagem para ser considerada nesse paradigma. Para isso, a linguagem precisa atender a quatro tópicos bastante importantes: abstração, encapsulamento, herança e polimorfismo.

Abstração

A abstração consiste em um dos pontos mais importantes dentro de qualquer linguagem Orientada a Objetos. Como estamos lidando com uma representação de um objeto real (o que dá nome ao paradigma), temos que imaginar o que esse objeto irá realizar dentro de nosso sistema. São três pontos que devem ser levados em consideração nessa abstração. O primeiro ponto é darmos uma identidade ao objeto que iremos criar. Essa identidade deve ser única dentro do sistema para que não haja conflito. Na maior parte das linguagens, há o conceito de pacotes (ou

namespaces). Nessas linguagens, a identidade do objeto não pode ser repetida dentro do pacote, e não necessariamente no sistema inteiro.

A segunda parte diz respeito a características do objeto. Como sabemos, no mundo real qualquer objeto possui elementos que o definem. Dentro da programação orientada a objetos, essas características são nomeadas propriedades. Por exemplo, as propriedades de um objeto “Cachorro” poderiam ser “Tamanho”, “Raça” e “Idade”. Por fim, a terceira parte é definirmos as ações que o objeto irá executar. Essas ações, ou eventos, são chamados métodos. Esses métodos podem ser extremamente variáveis, desde “Acender()” em um objeto lâmpada até “Latir()” em um objeto cachorro.

#REFLITA#

Abstração em software é o que permite que os projetos sejam desenvolvidos no papel (ou mundo das ideias) antes de serem construídos. Dessa forma, é fundamental que o desenvolvedor de software tem capacidade de abstrair conceitos e combiná-los.

REFLITA

Encapsulamento

O encapsulamento é uma das principais técnicas que define a programação orientada a objetos (FREEMAN, 2007). Se trata de um dos elementos que adicionam segurança à aplicação em uma programação orientada a objetos pelo fato de esconder as propriedades, criando uma espécie de caixa preta.

A maior parte das linguagens orientadas a objetos implementam o encapsulamento baseado em propriedades privadas, ligadas a métodos especiais chamados *getters* e *setters*, que irão retornar e aplicar o valor da propriedade, respectivamente. Essa atitude evita o acesso direto a propriedade do objeto, adicionando uma outra camada de segurança à aplicação.

Para fazermos um paralelo com o que vemos no mundo real, temos o encapsulamento em outros elementos. Por exemplo, quando clicamos no botão ligar da televisão, não sabemos o que está acontecendo internamente. Podemos então dizer que os métodos que ligam a televisão estão encapsulados.

#REFLITA#

O funcionamento interno de um objeto deve ser conhecido somente por ele mesmo. Isso evita propagação de erros e facilita a manutenção do software.

REFLITA

Herança

O reuso de código é uma das grandes vantagens da programação orientada a objetos (WAZLAWICK, 2014). Muito disso se dá por uma questão que é conhecida como herança. Essa característica otimiza a produção da aplicação em tempo e linhas de código. Para entendermos essa característica, vamos imaginar uma família: a criança, por exemplo, está herdando características de seus pais. Os pais, por sua vez, herdaram algo dos avós, o que faz com que a criança também o faça, e assim sucessivamente. Na orientação a objetos, a questão é exatamente assim,

como mostra a Figura 2. O objeto abaixo na hierarquia irá herdar características de todos os objetos acima dele, seus *ancestrais*. A herança a partir das características do objeto mais acima é considerada herança direta, enquanto as demais são consideradas heranças indiretas. Por exemplo, na família, a criança herda diretamente do pai e indiretamente do avô e do bisavô.

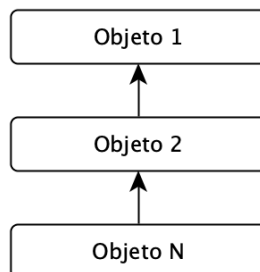


Figura 2 – Herança na Orientação a Objeto

A questão da herança varia bastante de linguagem para linguagem. Em algumas delas, como C++, há a questão da herança múltipla. Isso, essencialmente, significa que o objeto pode herdar características de vários ancestrais ao mesmo tempo diretamente. Em outras palavras, cada objeto pode possuir quantos pais for necessário. Devido a problemas, essa prática não foi difundida em linguagens mais modernas, que utilizam outras artimanhas para criar uma espécie de herança múltipla.

Outras linguagens orientadas a objetos, como C#, trazem um objeto base para todos os demais. A classe *object* fornece características para todos os objetos em C#, sejam criados pelo usuário ou não.

#ATENÇÃO#

Evite a utilização de herança com muitos objetos. Pode parecer tentador a ideia de obter códigos de outros objetos acima. Entretanto, quanto maior a hierarquia, mais difícil a manutenção do software.

#ATENÇÃO#

Polimorfismo

Outro ponto essencial na programação orientada a objetos é o chamado polimorfismo. Na natureza, vemos animais que são capazes de alterar sua forma conforme a necessidade, e é dessa ideia que vem o polimorfismo na orientação a objetos. Como sabemos, os objetos filhos herdam as características e ações de seus ancestrais. Entretanto, em alguns casos, é necessário que as ações para um mesmo método seja diferente. Em outras palavras, o polimorfismo consiste na alteração do funcionamento interno de um método herdado de um objeto pai.

Como um exemplo, temos um objeto genérico “*Eletrdoméstico*”. Esse objeto possui um método, ou ação, “*Ligar()*”. Temos dois objetos, “*Televisão*” e “*Geladeira*”, que não irão ser ligados da mesma forma. Assim, precisamos, para cada uma das classes filhas, reescrever o método “*Ligar()*”.

Com relação ao polimorfismo, valem algumas observações. Como se trata de um assunto que está intimamente conectado à herança, entender os dois juntamente é uma boa ideia. Outro

ponto é o fato de que as linguagens de programação implementam o polimorfismo de maneiras diferentes. O C#, por exemplo, faz uso de método virtuais (com a palavra-chave *virtual*) que podem ser reimplementados (com a palavra-chave *override*) nas classes filhas. Já em Java, apenas o atributo “*@Override*” é necessário.

Esses quatro pilares são essenciais no entendimento de qualquer linguagem orientada a objetos e da orientação a objetos como um todo. Cada linguagem irá implementar esses pilares de uma forma, mas essencialmente é a mesma coisa. Apenas a questão da herança, como comentado, que pode trazer variações mais bruscas, como a presença de herança múltipla. Além disso, o encapsulamento também é feito de maneiras distintas nas diversas linguagens, embora os *getters* e *setters* sejam praticamente onipresentes.

Conceitos fundamentais de análise e projetos OO

Entendido os conceitos do que vem a ser Orientação a Objeto (OO), vamos analisar como utilizar os mesmos na Análise e Projetos Orientado a Objetos (APOO). Existem várias definições de APOO. Vamos considerar duas inicialmente, uma do ponto de vista de fases no ciclo de vida de desenvolvimento de software e do ponto de vista de partes interessadas no desenvolvimento de software.

Vamos a primeira alternativa. A análise modela o problema e consiste das atividades necessárias para entender o domínio do problema (o que deve ser feito). Sendo, portanto, uma atividade de investigação (GAMMA, 2000). O projeto modela a solução e consiste das atividades de criação (como pode ser feito). Dessa forma, uma atividade de engenharia (GAMMA, 2000). A Figura 3 demonstra tal alternativa.

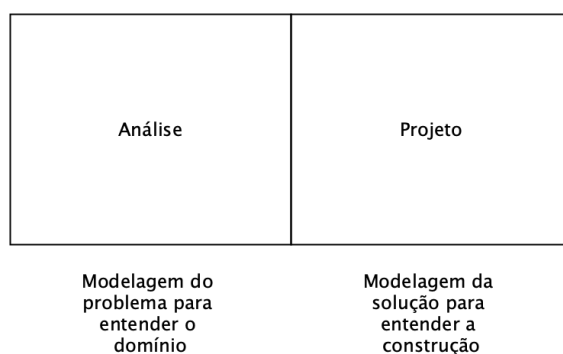


Figura 3 – Modelagem do problema e da solução

#ATENÇÃO#

Existe uma enorme confusão no desenvolvimento de software em relação a essas atividades. A análise envolve em grande parte investigação do problema. Aqui mora o perigo. Desenvolvedores de software tem uma tendência a modelar soluções com um baixo entendimento do problema a ser resolvido gerando, portanto, soluções boas para problemas que não foram de fato solucionados.

#ATENÇÃO#

Segunda alternativa. A análise consiste de todas as atividades feitas com ou para o conhecimento do usuário (LARMAN, 2007). A informação produzida é aquela que o usuário deve discutir e

aprovar. O projeto inclui as atividades que resultam em informação que interessa apenas ao pessoal que vai construir a solução. Com essa definição, a análise invade um pouco o "lado da solução", pois o cliente deve discutir alguns tipos de interações que ocorrerão na interface do usuário, etc. A Figura 4 demonstra tal alternativa.

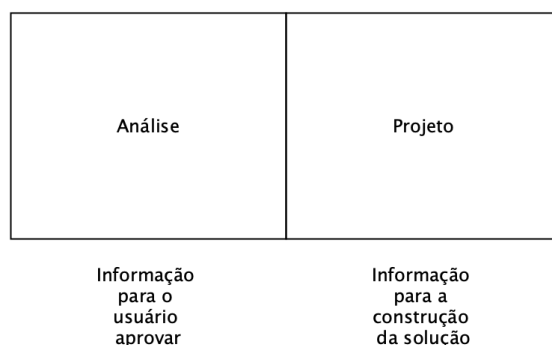


Figura 4 – Informação importante para o usuários e desenvolvedores

Observe portanto que não há definição binária que isole "análise" de "projeto". Neste estudo, adotaremos a segunda alternativa, pois queremos associar as palavras "análise" e "projeto" aos artefatos entregues no final de cada fase. Um modelo de análise deve ser aprovado pelo cliente e pode incluir alguma (pequena) discussão da solução, principalmente no que diz respeito à interface com usuário, etc.

#ATENÇÃO#

Esse avanço da informações de análise para dentro do projeto e vice-versa pode acontecer em todas as etapas do ciclo de desenvolvimento de software. Aqui ilustramos dessa forma para facilitar o entendimento, mas na vida real, as coisas tendem a ser um pouco mais complexas.

#ATENÇÃO#

Adicionalmente, podemos ter as demais etapas da fabricação de um software como a construção, etc. Isso pode ser visto da Figura 5 abaixo.

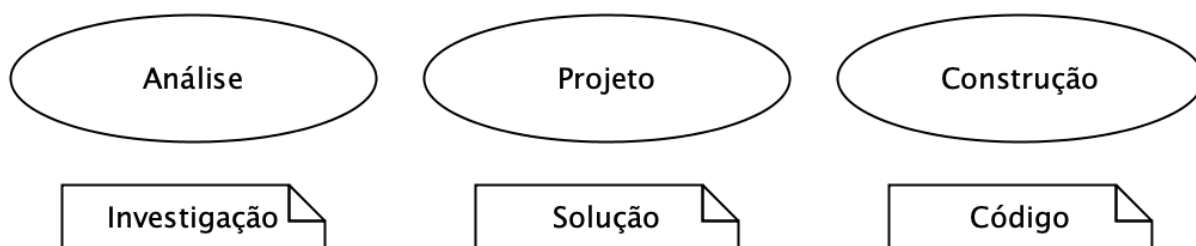


Figura 5 – Fases de desenvolvimento de um software

Neste caso, o que é Análise e Projeto Orientados a Objeto (APOO)? A perspectiva empregada é de objetos (coisas, conceitos ou entidades). Durante a Análise OO com ênfase em achar e descrever objetos (ou conceitos) no domínio do problema. Por exemplo, num sistema de informação para uma biblioteca, alguns dos conceitos são *Livro*, *Biblioteca*, *Usuário*. Tais objetos podem ter atributos e responsabilidades.

Já durante o projeto orientado a objeto, a ênfase está em achar objetos lógicos de software que poderão ser eventualmente implementados usando uma linguagem de programação OO. Tais objetos podem ter atributos e métodos. Cuidado com algumas questões, como:

- Não é verdade que haja correspondência 1-para-1 entre entidades no modelo de análise e entidades no modelo de projeto
- Pode haver entidade do modelo de análise que não será representado no Projeto (é raro)
- Pode haver entidade adicional no projeto (é frequente), por exemplo: Conexão de banco de dados, objeto controlador, cache de objetos, etc.

Durante a construção (programação OO), os objetos do projeto são implementados e testados. Além disso, outras questões de engenharia de software precisam ser consideradas como o local de armazenamento dos códigos produzidos, a compilação dos códigos e disponibilização para os usuários finais. A Figura 6 abaixo mostra um exemplo de como ficaria os resultado de cada fase, desde a análise até a construção.

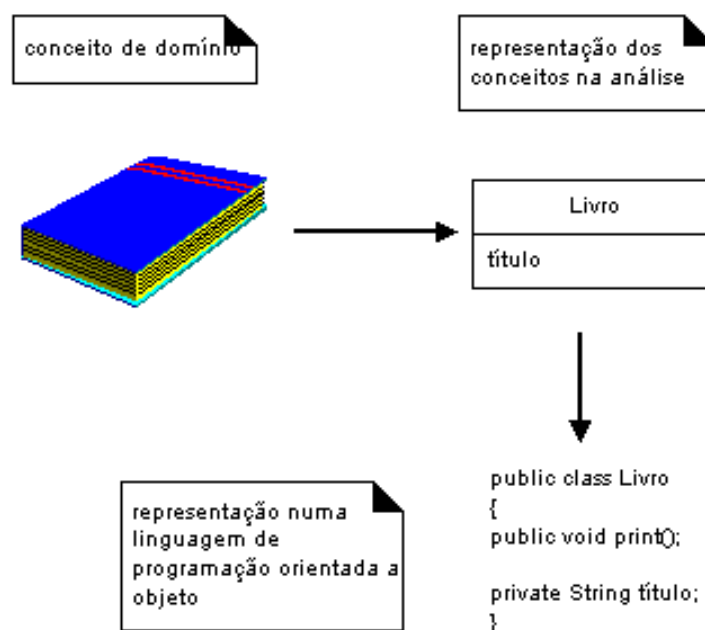


Figura 6 – Domínio, solução e construção

Vale ressaltar que as classes de solução geralmente são classes do domínio e classes de infraestrutura. Sempre que possível é interessante ter essas classes componentizadas pois são tarefas comuns.

Iterativo, Incremental e Ágil

Vamos entender o que vem a ser o modelo iterativo e o modelo incremental. No modelo de processo de software incremental tem-se a ideia de se desenvolver uma implementação inicial,

apresentá-la ao usuário e evoluir o software ao longo do tempo de acordo com o feedback do usuário (WAZLAWICK, 2014). Portanto, especificação, desenvolvimento e validação ocorrem em paralelo para cada um dos incrementos do software que estão em construção. A Figura 7 abaixo apresenta a forma de desenvolvimento incremental.

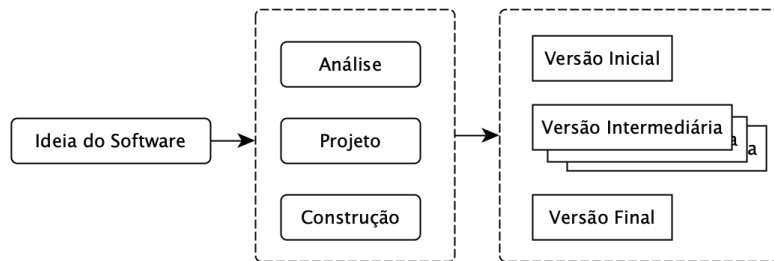


Figura 7 – Modelo incremental

No modelo iterativo, a cada iteração uma ou mais atividades são executadas antes de se iniciar uma nova atividade (FREEMAN, 2007). Ou seja, o fluxo de execução do processo de desenvolvimento ocorre desta maneira, conforme a Figura 8 abaixo:

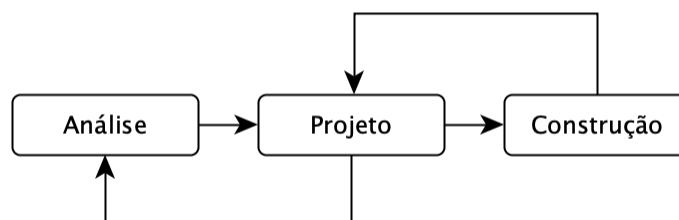


Figura 8 – Modelo iterativo

O modelo iterativo muitas vezes é confundido com o modelo incremental. Um processo de desenvolvimento de software é incremental quando a cada rodada é desenvolvido um pedaço inteiro do software. Já no iterativo, a cada iteração, se avança no conhecimento do projeto, novos requisitos são elicitados e a arquitetura do software é revisada. No modelo incremental, necessariamente teremos entrega de software ao término de uma iteração, o que não é verdade para o modelo iterativo. A Figura 9 a seguir esclarece qualquer dúvida que tenha persistido.

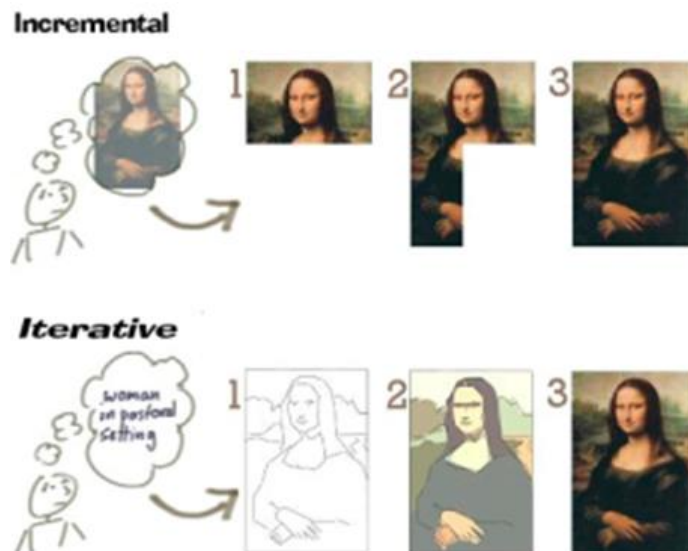


Figura 9 – Modelo iterativo e incremental

Então, se o Leonardo da Vinci tivesse desenhado um pedaço do quadro da Mona Lisa de cada vez, o pintor teria realizado um desenvolvimento incremental. Mas, na verdade, ele começou desenhando um esboço, depois preencheu as áreas com tinta e depois pintou os detalhes, ou seja, ele desenvolveu iterativamente o quadro da Mona Lisa. Ele foi conhecendo e especificando os requisitos da Mona Lisa cada vez mais a cada iteração.

O detalhe é que os modelos iterativo e incremental não são excludentes. O RUP, por exemplo, é um processo de desenvolvimento de software iterativo e incremental. O processo iterativo é aquele que progride através de refinamentos sucessivos. A equipe de desenvolvimento apresenta uma parte de um sistema, ciente de que está incompleto ou deficitário em algumas (talvez muitas) partes. A equipe então refina estas partes até o produto apresentar um nível satisfatório. A cada iteração, o software é melhorado através da adição de um maior nível de detalhe (LARMAN, 2007).

#REFLITA#

Iterativo e incremental contribui muito para o desenvolvimento de softwares com requisitos não bem compreendidos no início do projeto.

REFLITA

Por exemplo, numa primeira iteração, uma tela de pesquisa pode ser codificada para suportar apenas o tipo mais simples de pesquisa. A segunda iteração pode adicionar critérios de pesquisa adicional. Finalmente, uma terceira iteração pode adicionar tratamento de erros. Uma boa analogia, é escultura. Em primeiro lugar, o escultor seleciona uma pedra de tamanho apropriado. Em seguida, escultor vai esculpir a forma geral na pedra. Neste ponto, pode-se talvez distinguir a cabeça e o torso, e discernir que o trabalho final será o de um corpo humano e não de um pássaro. Em seguida, o escultor refina seu trabalho adicionando detalhes. No entanto, o escultor não vai ter nenhuma área completa até todo o trabalho estar completo.

O processo incremental é aquele em que o software é construído e entregue em pedaços. Cada peça, ou incremento representa um subconjunto completo de funcionalidades. O incremento pode ser pequeno ou grande, talvez variando de uma simples tela de *login* a um conjunto

altamente flexível de telas de gerenciamento de dados. Cada incremento é totalmente codificado e testado, e a expectativa comum é que o trabalho de uma iteração não precisará ser revisitado. Um escultor incremental iria pegar uma parte do trabalho e se concentrar inteiramente sobre ele até que seja concluído. Ele pode selecionar pequenos incrementos (primeiro o nariz, olhos, boca e assim por diante) ou grandes incrementos (cabeça, tronco, pernas e depois os braços). No entanto, independentemente do tamanho do incremento, o escultor incremental tende a terminar esse trabalho completamente. Scrum e Ágil são tanto incrementais, quanto iterativos. Eles são iterativos quando se planeja o trabalho de uma iteração, para ser melhorado em iterações subsequentes. Eles são incrementais porque o trabalho concluído é entregue ao longo do projeto.

#REFLITA#

Incrementos de software melhoram a visibilidade e aumentam a confiança da equipe para continuar entregando software de qualidade. Melhora também os feedbacks dos usuários e seu engajamento ao projeto.

REFLITA

Para melhor ilustrar as diferenças entre iterativo e incremental, vamos considerar a construção de um site de forma *iterativa*, mas não de forma incremental. Para fazer isso, a equipe iria construir um pouco de cada parte do site – alteração de perfil, busca, anúncios, etc. A equipe, então, vai rever todas as partes, melhorando cada uma delas. A equipe deve rever todas as partes, aplicando as melhorias, desta forma todo o site está ficando um pouco melhor.

Em seguida, vamos considerar o desenvolvimento do mesmo site com uma abordagem puramente *incremental*, não um processo iterativo. Se um site de namoro fosse construído de forma incremental, o time iria construir um perfeito sistema de gerenciamento do perfil antes de começar qualquer outra parte do site. Depois, eles então, iriam construir a segunda área – pesquisa – e passariam para a terceira área. Cada área funcional concluída deve ficar perfeita, antes de se iniciar a próxima.

Nem iterativo e nem incremental são bons isoladamente. Mas juntos – como eles estão no Scrum – são fantásticos. Dessa forma, ao trabalhar com APOO é importante utilizar algum modelo iterativo e incremental pois a união desse dois mundo facilita a entrega dos softwares e aumenta a qualidade dos mesmo.

Considerações Finais

Projetar software de qualidade é uma tarefa difícil (LARMAN, 2007). Mesmo que tenhamos as melhores ferramentas, processos e técnicas sempre devemos nos atentar para os princípios que norteiam esse tipo de trabalho. Alguns desses princípios são os padrões de projetos. Todos os padrões então apoiados em dois conceitos conhecidos como acoplamento e coesão. Ambos visam reduzir os efeitos de propagação de efeitos colaterais indesejáveis e melhorando o requisito não funcional de manutenibilidade.

UNIDADE 2

ANÁLISE

Objetivos de aprendizagem

- Aprender a etapa de concepção do projeto orientado a objetos
- Aprender como gerenciar requisitos evolutivos
- Aprender como gerenciar casos de uso

Introdução

Vivemos num mundo complexo marcado por transformações constantes, que força os diferentes tipos de mindset a tomar decisões rápidas e a se adaptar para sobreviver. Essa dinâmica pode ser representada por um acrônimo, incorporado ao jargão da defesa, que traduz este cenário: o “VUCA” – volatility, uncertainty, complexity e ambiguity (volatilidade, incerteza, complexidade e ambiguidade). O termo pode ser apropriado para a gestão de projetos, que muitas vezes lida com problemas também voláteis, incertos, complexos e ambíguos.

#REFLITA#

Existe muita pesquisa para entender o cenário VUCA no qual estamos atualmente. Isso pode mudar muito a forma como entendemos o mundo.

REFLITA

Neste contexto, as pressões por resultados rápidos e aquisições de tecnologia por oportunidade fazem com que as etapas iniciais do ciclo de vida para a realização de um projeto sejam negligenciadas. E as consequências podem ser nefastas, desde gastos muitas vezes superiores aos originalmente previstos, até a completa inviabilização de um projeto, com a criação dos chamados “elefantes brancos”.

Diante dessa crescente complexidade, tornou-se de vital importância a redução do risco associado às etapas de um projeto. Para tanto, antes de iniciá-lo, faz-se necessária uma análise minuciosa do seu ciclo de vida, que compreende as etapas de concepção, requisitos, arquitetura, desenvolvimento/produção e operação, até a sua retirada de serviço/descarte.

Essa estratégia é crucial no âmbito dos projetos complexos, no qual é necessária a visão sistêmica, materializada na metodologia de systems engineering – que vai muito além da visão técnica. Aplicada numa visão de Sistemas de Sistemas, apresenta-se como a metodologia ideal para a concepção e o desenvolvimento de projetos complexos, pois analisa de forma integrada todos os componentes e partes interessadas. Desta forma, é obtido um resultado global melhor do que aquele que seria obtido ao aplicar a visão tradicional de gestão.

Trata-se de uma abordagem holística, com base em uma visão de alto nível, que permite implementar e analisar sistemas compostos por elementos heterogêneos, complexos e independentes, para que se integrem de forma cooperativa e eficaz. Mas, para que haja êxito, é necessário ter especial foco nas etapas iniciais de concepção e resistir a se ater exclusivamente à etapa de desenvolvimento e produção.

Concepção

A fase de planejamento é o processo fundamental de compreensão por que um sistema de informação deve ser construído e determinar como a equipe do projeto o construirá. Tem dois passos:

- I) Durante o início do projeto, o valor de negócios do sistema para a organização é identificado: como é que vai reduzir os custos ou aumentar receitas? A solicitação do sistema apresenta um breve resumo de uma necessidade de negócio, e explica como um sistema que suporta a necessidade criará valor aos negócios. O departamento de Sistema de Informação trabalha em conjunto com a pessoa ou departamento que gerou o pedido para realizar uma análise de viabilidade. A viabilidade analisa os aspectos fundamentais do projeto proposto:
 - Viabilidade técnica da ideia (Podemos construí-lo?) Viabilidade econômica (Será que vai fornecer o valor do negócio?)
 - Viabilidade organizacional (Se nós o construirmos, ele vai ser usado?)
- O pedido do sistema e a análise de viabilidade são apresentados a uma comissão de aprovação, que decide se o projeto deve ser empreendido.
- II) Uma vez que o projeto seja aprovado, entra em gerenciamento de projetos. Durante a gestão de projetos, o gerente de projeto cria um plano de trabalho, equipes do projeto. O produto para gerenciamento de projetos é um plano de projeto, que descreve como a equipe do projeto vai começar a desenvolver o sistema.

Para ilustrar a importância das etapas iniciais do ciclo de vida dos projetos, vamos trazer como referência um estudo elaborado pelo Departamento de Defesa dos EUA (DoD). A partir do histórico de diversos projetos executados, o DoD realizou um estudo dos custos durante o seu ciclo de vida, que é apresentado no gráfico a seguir. Especial atenção foi dada ao momento no qual são tomadas decisões que definem os custos do projeto e também o impacto de correções aplicadas, quando consideradas em diferentes momentos do seu ciclo de vida.

Nesse gráfico, o custo total inclui os custos incorridos desde a sua concepção até a sua retirada de serviço, de forma acumulativa. A etapa de concepção, requisitos e arquitetura, conforme indicado no gráfico, representa 20% do custo acumulado total do projeto.

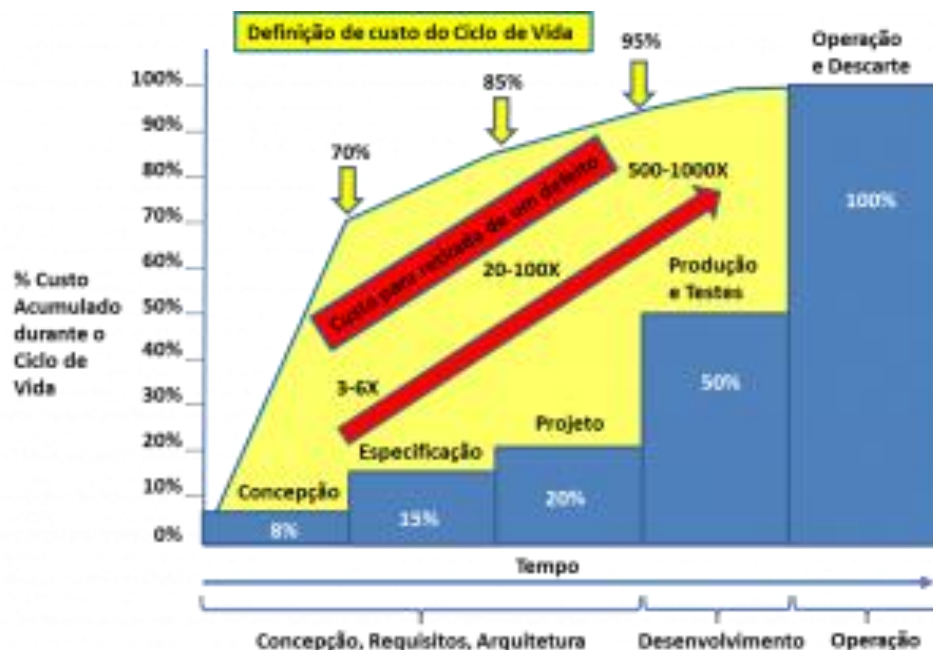


Figura 10 – Adaptado de Incoose Systems Engineering Handbook

O gráfico ainda destaca os momentos nos quais são definidos os custos, representados na curva amarela. Esses momentos representam a definição das tecnologias a serem utilizadas, os meios de integração a outros sistemas, aspectos logísticos, capacidades humanas e de instalações requeridas para a sua entrada em operação, manutenção e evolução ao longo da sua vida útil, até a retirada de serviço. Através do gráfico pode-se observar que somente a etapa de concepção define 70% dos custos do ciclo de vida de um projeto/programa. Outros 25% são definidos nas etapas de requisitos e arquitetura. Assim, 95% dos custos são definidos antes do início da produção. Significativo, não?

#REFLITA#

Existe uma preocupação com o descarte de um produto criado. Em outras palavras, isso é chamado de engenharia reversa. Software também possui isso em um certo nível. Pense em termos de como retirar um software de operação.

REFLITA

Esses aspectos tomam uma dimensão mais importante quando pensamos que uma definição malfeita ou um requisito incorreto irá representar uma correção de rumo do projeto mais adiante. Esses ajustes implicam custos para a correção de um defeito ou erro de especificação do sistema. Tomando como referência um valor simbólico “1” para o custo de correção de um defeito detectado durante a etapa de concepção, esse custo será exponencialmente maior caso o mesmo defeito seja detectado em etapas posteriores do ciclo de vida. Caso esse defeito seja detectado na fase de produção, o custo para a sua correção será da ordem de 500 a 1000 vezes maior do que se fosse detectado na fase de concepção. Isso representa um custo oculto no ciclo de vida de um projeto que pode levá-lo a nunca ser concluído e certamente terá como consequência um custo

final muito maior do que o originalmente previsto no orçamento. Portanto, ainda que tenhamos inputs considerados urgentes, devemos ter o devido cuidado com as fases iniciais de definição de uma nova solução ou sistema, e trabalharmos com a metodologia de *systems engineering* para maior eficácia dos projetos.

Requisitos Evolutivos

Software tem uma característica diferente de outros produtos de engenharia: seus requisitos são evolutivos. Antes de iniciarmos esse estudo, vamos para algumas definições para o melhor entendimento do assunto. O que vem a ser um requisito? Uma das definições aceitas é que um requisito é uma condição ou capacidade que um software deve atender (LARMAN, 2007).

Software possui também um outro tipo de requisito chamado de Requisito não Funcional que está relacionado com as condições no qual uma funcionalidade deve processar. Por exemplo, a funcionalidade poderia ser o acesso via usuário e senha. Isso seria um requisito funcional. Agora, se isso vir acompanhado de um armazenamento de senha com mais de 10 caracteres sendo criptografados via chave SHA256. Esse último requisito é uma condição para que o primeiro seja executado.

Existe uma forma de criar uma pirâmide ligando as necessidades dos usuários por meio de requisitos e demais técnicas de análise. Vamos entender essa pirâmide. Ela é composta dos elementos:

- Necessidades de usuários: requisitos que são demandados por usuários ou especialistas que conhecem do domínio do problema que o software deve atender
- Funcionalidade: serviços que devem ser fornecidos pelo software para os usuários
- Casos de uso: descrição do comportamento de uma funcionalidade em uma sequência de passos
- Cenários de casos de uso: maneiras de utilização dos casos de uso desenvolvidos
- Requisitos suplementares: esse é um outro nome para requisitos não funcionais
- Casos de testes: especificação de cenários de testes com os resultados esperados para cada cenário de caso de uso

Podemos verificar como essa pirâmide funciona, vamos analisar a Figura 11 abaixo.

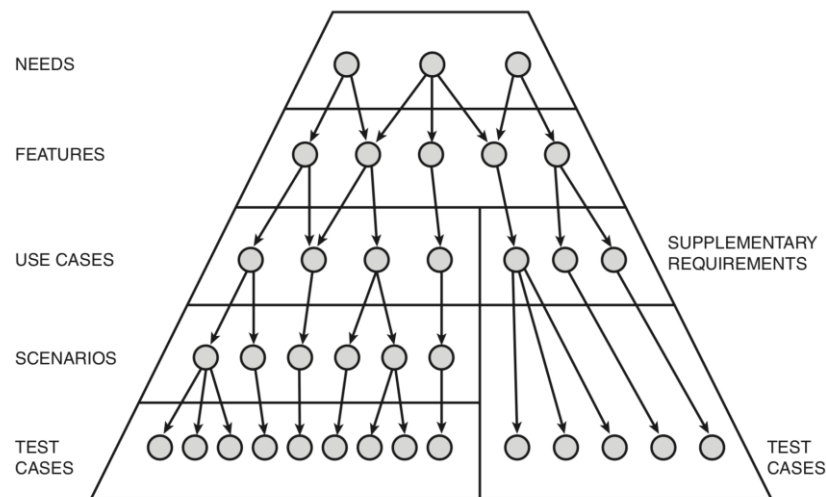


Figura 11 – A Pirâmide dos Requisitos

Com isso podemos entender que os requisitos podem ser decompostos de maneira estruturada e com vários artefatos gerados ao longo do desenvolvimento do software.

Existem algumas características que devem ser observadas para obter um requisito de boa qualidade (ENGHOLM, 2017). Bons requisitos deveriam ter as seguintes características:

- Não ambíguo: isso quer dizer que os requisitos não podem ter dupla implementação pois isso dificulta o entendimento e a comunicação dos mesmos
- Testável: um requisito precisa ser testado para que o mesmo possa ser implementado e testado de maneira objetiva
- Claro: não basta ser não ambíguo, ele precisa ser de simples entendimento, sem a utilização de recursos de linguagem desnecessários
- Correto: o requisito deve expressar a necessidade dos usuários da maneira mais próxima possível da realidade
- Viável: o requisito precisa ser possível de implementar com os recursos financeiros, de tempo, e demais existentes
- Livre de tecnologia: os requisitos inicialmente não devem se ater a questões tecnológicas (exceto por requisitos de viabilidade) para não limitar a solução de maneira desnecessária
- Necessário: requisitos tem um custo para serem desenvolvidos, dessa forma, os mesmos devem ser considerados como necessários para serem implementados
- Contido: um requisito deve realizar uma única função de maneira mais independente possível

Com isso, entendemos que os requisitos podem ser decompostos ao longo da análise e quais são as características que expressam a qualidade de um bom requisito.

#ATENÇÃO#

Erros no desenvolvimento de requisitos costumam ser baratos para correção durante a fase de análise. Entretanto, conforme os mesmos vão sendo enviados para as demais fases (como projeto, construção, testes, operação) vai ficando mais caro para resolver eventuais defeitos.

#ATENÇÃO#

Casos de Uso

Os casos de uso são uma parte muito importante da análise orientada a objeto (AOO). Se não for o mais importante. O caso de uso permeia toda a análise e serve de base para a implementação de códigos e testes de software. Por isso, vamos estudar as nuances dos casos de uso como o fundamento da AOO. A Figura 12, abaixo, mostra como os casos de uso atravessam várias fases no desenvolvimento de software.

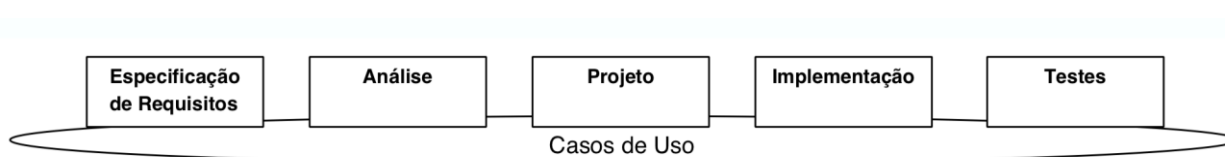


Figura 12 – A Importância do Caso de Uso na Análise Orientada a Objeto

Primeiramente, vamos a uma definição do que vem a ser um caso de uso. Na Engenharia de Software, um caso de uso (do inglês use case) é um tipo de classificador representando uma unidade funcional coerente provida pelo sistema, subsistema, ou classe manifestada por sequências de mensagens intercambiáveis entre os sistemas e um ou mais atores (LARMAN, 2007). Os casos de uso são compostos de dois elementos básicos: (a) o diagrama de caso de uso e o (b) descritivo de caso de uso. Vamos estudar com detalhes esses elementos abaixo .

Diagrama de caso de uso

O diagrama de caso de uso descreve a funcionalidade proposta para um novo sistema que será projetado, é uma excelente ferramenta para o levantamento dos requisitos funcionais do sistema (WAZLAWICK, 2014). Ele apresenta uma visão geral das funcionalidades que um software deve implementar. A Figura 13, abaixo, apresenta esse diagrama.

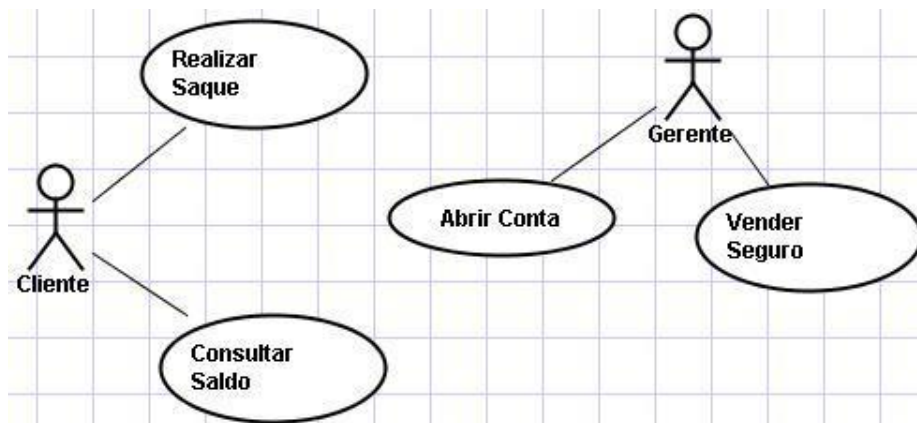


Figura 13 – O Diagrama de Caso de Uso

Na Figura 13 podemos ver 3 elementos básicos de um diagrama de caso de uso: (a) o ator, (b) o caso de uso e (c) a associação entre o ator e o caso de uso. Vamos entender um pouco melhor cada um desses elementos agora.

Ator

Um ator representa alguma coisa que interage com o caso de uso. Via de regra ele é uma função assumida por um usuário de um software. No exemplo da Figura 13 temos 2 atores, o cliente e o gerente. O importante neste caso é mapear a função do usuário que pode ser caracterizada como um perfil de utilização do software. Adicionalmente, um ator pode ser um outro software, uma API, uma máquina. Isso é importante pois nem sempre o execução de um caso de uso depende de uma pessoa, pode ser uma rotina agendada.

#REFLITA#

Quais seriam os possíveis atores que os softwares que você conhece que poderiam ser representados por atores não humanos.

REFLITA

Caso de Uso

Um caso de uso representa uma funcionalidade a ser provida pelo software implementado. Neste caso, temos quatro funcionalidades sendo que o comportamento das mesmas são representados pelos casos de uso. Dessa forma, podemos verificar que cada ator executa dois casos de uso. Um ponto muito importante desse diagrama está na possibilidade de visualização de todas as funcionalidades a serem implementadas e quais são os usuários que as executam.

#REFLITA#

Uma boa separação de quem executa qual caso de uso ajuda muito a criar os perfis de acesso de usuários do software a ser desenvolvido.

REFLITA

Associação

A associação representa a forma de interação entre ator e caso de uso. Existem diferentes tipos de associação. Por exemplo, temos um ator que pode executar varias vezes um caso de uso, uma vez por dia, etc. Por enquanto vamos nos manter com a mais simples de todas as associações representadas no diagrama acima.

Descritivo de caso de uso

O descritivo de um caso de uso apresenta passo a passo como o mesmo deve ser executado. Há alguns cuidados a ter para ter a certeza que um caso de uso está corretamente compreendido. Habitualmente é adoptado um standard que requer o preenchimento de alguns campos relativos ao caso de uso de modo a facilitar o trabalho em grupo e a clareza do relacionamento entre vários casos de uso, e do caso de uso em relação aos atores e ao próprio sistema. Algumas das secções habitualmente utilizadas incluem:

- **Nome:** Identificador inequívoco do caso de uso, deve ser escrito em formato de verbo/substantivo e ser suficiente para o utilizador perceber a que se refere o caso de uso.
- **Iteração ou estado de desenvolvimento:** Descrição do estado atual do caso de uso à medida que este vai evoluindo.
- **Sumário:** Curto resumo do caso de uso.
- **Pré-condições:** Listagem das condições que se devem verificar quando o utilizador inicia este caso de uso. Não incluem triggers.
- **Triggers:** Eventos que ocorrem dando início ao caso de uso. Podem ser externos, temporais ou internos.
- **Linha de Eventos:** Esta secção descreve o curso de eventos ou cenário que se realiza. Usualmente é descrito através de uma sequência de eventos numerados.
- **Percursos Alternativos:** Descrição de percursos alternativos à linha de eventos básica.
- **Pós-condições:** Descrição do estado do sistema após a execução do caso de uso
- **Regras de negócio:** Secção reservada para informação adicional relativa à política da empresa ou restrições impostas pelo tipo de negócio.
- **Notas:** Informação adicional relativamente ao caso de uso, não coberta pelas secções anteriores.
- **Autor e data:** Listagem dos autores e datas das várias versões revistas.

Vamos ver um exemplo de caso de uso na prática. É importante entender os elementos envolvidos e para que o mesmo serve.

Nome: Pesquisar Online

Descrição: O pesquisador realiza N pesquisas com uma string de consulta

Ator: Pesquisador

Pré-condições: String de consulta válida (RN01)

Fluxo Principal

Fluxo Alternativo

Fluxo Exceção

Pós-condições: Exibir os resultados da consulta realizada

Regras: RN02, RN04

Figura 14 – O Descritivo de Caso de Uso

Pode-se notar que os elementos do caso de uso, como pré, pós-condições são sempre escritos de forma objetiva e direta. Como a língua portuguesa dá muita margem a interpretação é interessante reduzir isso com a estrutura do descritivo e com textos curtos. Um outro ponto importante é a separação das regras dos elementos do caso de uso. Podemos ver que as regras estão somente declaradas no caso de uso, mas não descritas. Dessa forma, podemos referenciá-las em outros locais sem a necessidade de reescrita. Essas regras são na forma IF-THEN. Vamos ver um exemplo.

RN01 - String de consulta válida

Se

(String.valor != nulo E String.valor != espaço)

Então

Válida

Senão

Inválida

Figura 15 – Um Exemplo de Regra de Negócio

Essas regras baseadas em IF-THEN são chamadas de sistemas especialistas no qual a especialidade do usuário é traduzida para o sistema a ser implementado. Grande parte dos problemas de software vem pelo não entendimento claro dessas regras e sua implementação

ineficiente. O que obriga a revisão quase que constante de código e aumenta a quantidade de bugs, caso não se tenha uma estratégia eficaz de testes.

Considerações Finais

Neste estudo entendemos como a necessidade dos usuários pode ser decomposta até os casos de uso. Entendemos também como os casos de uso permeiam o desenvolvimento de software durante a fase de projeto. Além disso, vimos com detalhes como funciona o diagrama de casos de uso e como funciona o descritivo de caso de uso. Lembrando um ponto importante que vem a ser as regras de negócio que devem ser separadas e baseadas em IF-THEN.

UNIDADE 3

PROJETO

Objetivos de aprendizagem

- Aprender sobre o diagrama de classes
- Aprender sobre o diagrama de atividades
- Aprender sobre o diagrama de maquina de estados

Introdução

Como vimos no capítulo anterior, a análise consiste em entender o domínio do problema. Neste caso, o projeto vem a ser o estudo da solução. Ou seja, uma vez entendido o problema, podemos passar para o entendimento da solução. Existem varias maneiras de se projetar a solução para um problema. Isso é conhecido como a variabilidade de soluções. Nosso objetivo nesta etapa é reduzir essa variabilidade para não consumir recursos tentando soluções que não sejam viáveis na prática. Para isso, vamos utilizar a *Unified Modeling Language* para decompor o a solução do problema em pontos de vistas diferentes. Isso é importante pois funciona como um exame médico feito por diferentes equipamentos. Dessa forma, algum detalhe que escapou em um exame pode ser evidenciado por outro.

Diagrama de Classes

Em engenharia de software, um diagrama de classes na *Unified Modeling Language* (UML) é um tipo de diagrama estático que descreve a estrutura de um sistema mostrando as classes do sistema, seus atributos, operações (ou métodos) e as relações entre esses objetos. O diagrama de classes é o principal bloco de construção da modelagem orientada a objetos. Ele é usado para modelagem conceitual geral da estrutura do aplicativo e para modelagem detalhada traduzindo os modelos em código de programação.

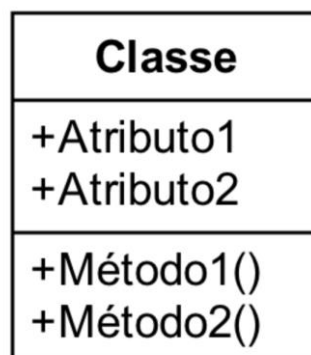


Figura 16 – Um Exemplo de Classe

Diagramas de classes também podem ser usados para modelagem de dados. As classes em

um diagrama de classes representam os elementos principais, as interações no aplicativo e as classes a serem programadas (PENDER, 2004).

No diagrama, as classes são representadas por caixas que contêm três compartimentos:

- O compartimento superior contém o nome da classe. É impresso em negrito e centralizado e a primeira letra é em maiúscula.
- O compartimento do meio contém os atributos da classe. Eles são alinhados à esquerda e a primeira letra é minúscula.
- O compartimento inferior contém as operações que a classe pode executar. Eles também são alinhados à esquerda e a primeira letra é minúscula.

No projeto de um sistema, várias classes são identificadas e agrupadas em um diagrama de classes que ajuda a determinar as relações estáticas entre elas. Com a modelagem detalhada, as classes do design conceitual são geralmente divididas em várias subclasses. Para descrever melhor o comportamento dos sistemas, esses diagramas de classes podem ser complementados por um diagrama de estados ou uma máquina de estados UML.

#REFLITA#

Como as entidades do mundo real podem ser convertidas para classes? Como o relacionamento dessas classes afeta o comportamento do software criado?

REFLITA

Visibilidade

Para especificar a visibilidade de um membro da classe (ou seja, qualquer atributo ou método), essas notações devem ser colocadas antes do nome do membro:

- + Público
- - Privado
- # Protegido
- ~ Pacote

A propriedade derivada é uma propriedade cujo valor (ou valores) é produzido ou calculado a partir de outras informações, por exemplo, usando valores de outras propriedades. A propriedade derivada é mostrada com seu nome precedido por uma barra invertida '/'.

Relacionamentos

Um relacionamento é um termo geral que abrange os tipos específicos de conexões lógicas encontradas em diagramas de classes e objetos. A UML define os seguintes relacionamentos:

Relacionamentos em nível de instância

Dependência

Uma dependência é uma conexão semântica entre elementos de modelo dependentes e independentes. Ele existe entre dois elementos se as alterações na definição de um elemento (o servidor ou o destino) puderem causar alterações no outro (o cliente ou a origem). Essa associação é unidirecional.

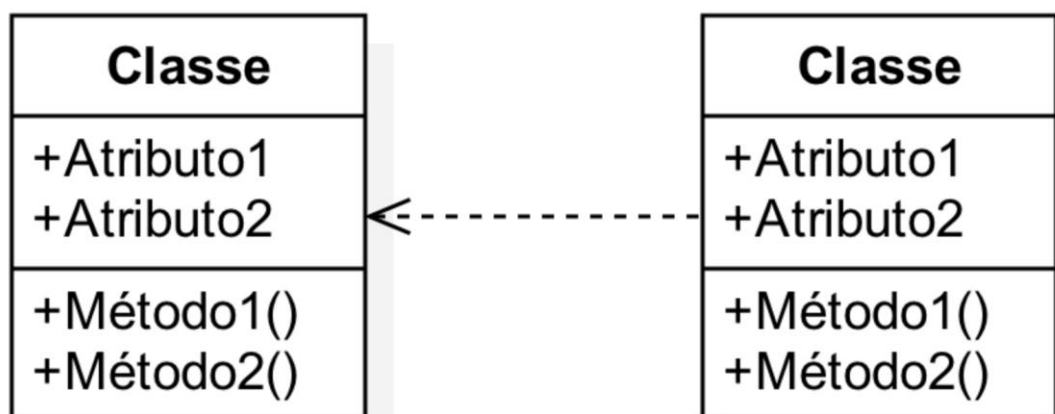


Figura 17 – Um Exemplo de Dependência entre Classes

Associação

Uma associação representa uma família de links. Uma associação binária (com duas extremidades) é normalmente representada como uma linha. Uma associação pode vincular qualquer número de classes. Uma associação com três links é chamada de associação ternária. Uma associação pode ser nomeada e os fins de uma associação podem ser adornados com nomes de função, indicadores de propriedade, multiplicidade, visibilidade e outras propriedades (PENDER, 2004).

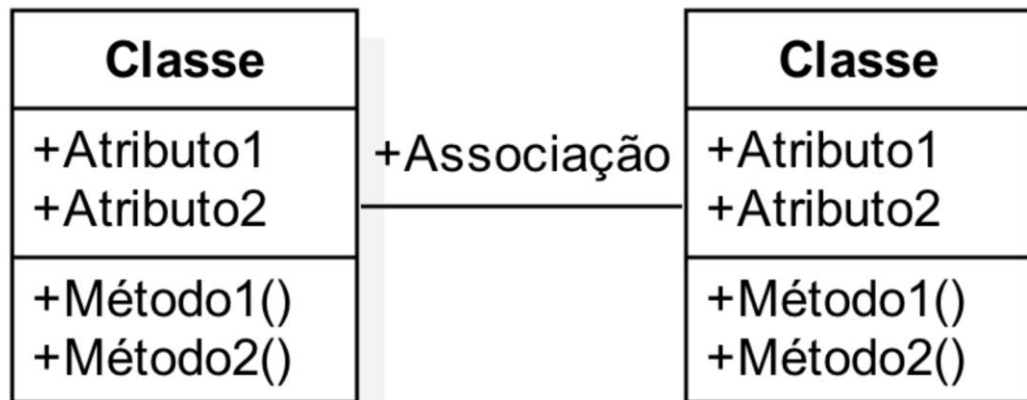


Figura 18 – Um Exemplo de Associação entre Classes

Existem quatro tipos diferentes de associação: bidirecional, unidirecional, agregação (inclui agregação de composição) e reflexiva. Associações bidirecionais e unidirecionais são as mais comuns. Por exemplo, uma classe de voo é associada a uma classe de plano bidirecionalmente. Associação representa o relacionamento estático compartilhado entre os objetos de duas classes.

Agregação

Agregação é uma variante do relacionamento de associação "tem um". É uma associação que representa um relacionamento parcial ou total. Como um tipo de associação, uma agregação pode ser nomeada e ter os mesmos adornos que uma associação pode. No entanto, uma agregação não pode envolver mais de duas classes; deve ser uma associação binária. Além disso, dificilmente há uma diferença entre agregações e associações durante a implementação, e o diagrama pode ignorar completamente as relações de agregação.

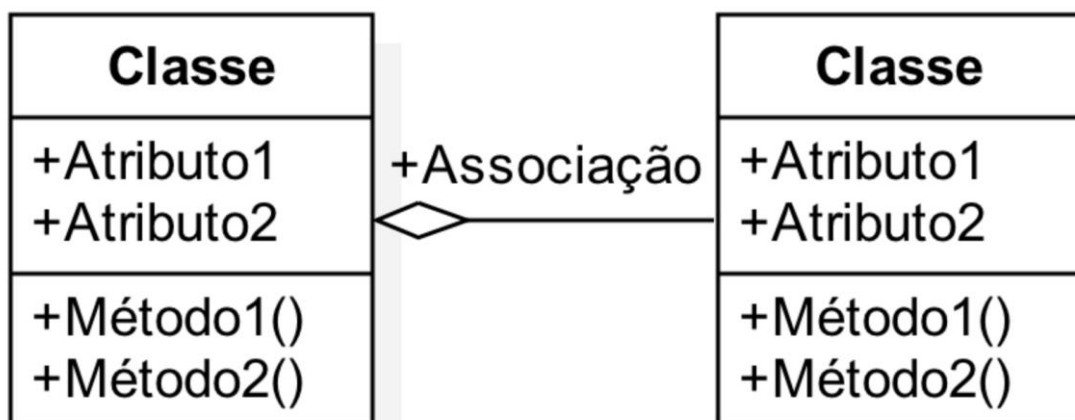


Figura 19 – Um Exemplo de Agregação entre Classes

A agregação pode ocorrer quando uma classe é uma coleção ou contêiner de outras classes, mas as classes contidas não possuem uma forte dependência de ciclo de vida no contêiner. O conteúdo do contêiner ainda existe quando o contêiner é destruído. Na UML, ela é representada graficamente como uma forma de losango oca na classe de contenção com uma única linha que a conecta à classe contida. O agregado é semanticamente um objeto estendido que é tratado como uma unidade em muitas operações, embora fisicamente seja feito de vários objetos menores. Exemplo: biblioteca e estudantes. Aqui o aluno pode existir sem biblioteca, a relação entre aluno e biblioteca é agregação (PENDER, 2004).

Composição

O diagrama no topo mostra Composição entre duas classes: um carro tem exatamente um carburador, e um carburador tem no máximo um carro (os carburadores podem não existir como partes separadas, separados de um carro específico). Essa é uma associação que as partes não existem sem o todo (PENDER, 2004).

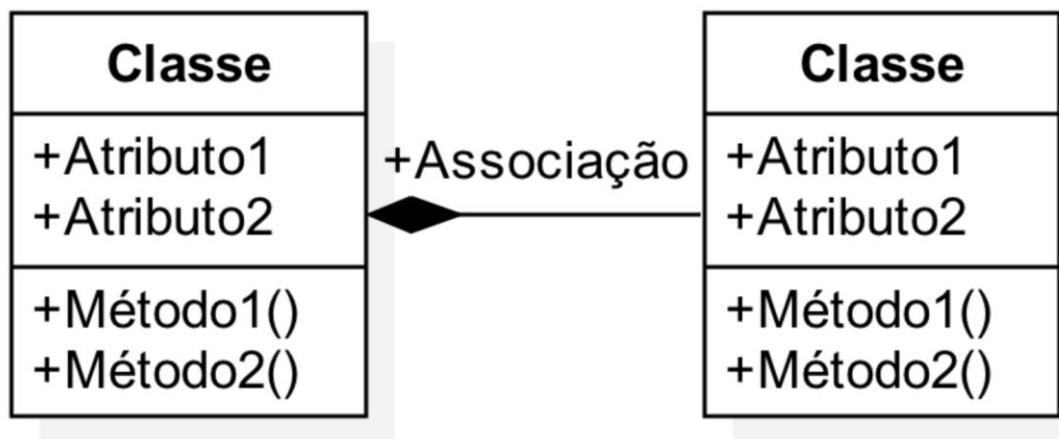


Figura 20 – Um Exemplo de Composição entre Classes

Diferenças entre Composição e Agregação

Relação de composição:

- Ao tentar representar relações de todo o mundo real, por exemplo um motor faz

parte de um carro

- Quando o recipiente é destruído, o conteúdo também é destruído, e. uma universidade e seus departamentos

Relacionamento de agregação:

- Ao representar uma relação de software ou banco de dados, por exemplo O modelo de carro ENG01 faz parte de um modelo de carro CM01, já que o motor ENG01 também faz parte de um modelo de carro diferente
- Quando o recipiente é destruído, o conteúdo geralmente não é destruído, e um professor tem alunos, quando o professor morre, os alunos não morrem junto com ele

Assim, a relação de agregação é frequentemente "contenção de catálogo" para distingui-la da contenção "física" da composição.

#REFLITA#

Por que existem vários tipos de relacionamento e como os mesmo podem ser capturados do mundo real?

REFLITA

Relacionamentos em nível de classe

Generalização / Herança

Indica que uma das duas classes relacionadas (a subclasse) é considerada uma forma especializada da outra (o tipo super) e a superclasse é considerada uma Generalização da subclasse. Na prática, significa que qualquer instância do subtipo é também uma instância da superclasse. Uma árvore exemplar de generalizações dessa forma é encontrada na classificação biológica: os humanos são uma subclasse de símios, que é uma subclasse de mamíferos, e assim por diante. A relação é mais facilmente entendida pela frase "um A é um B" (um humano é um mamífero, um mamífero é um animal).

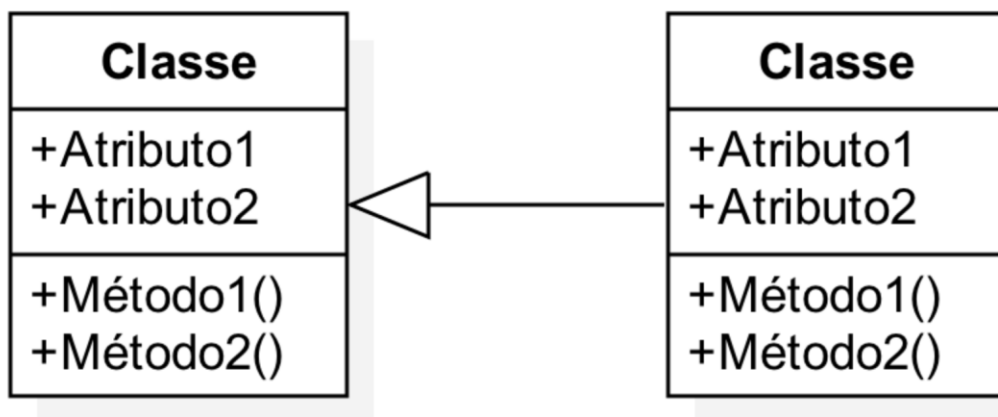


Figura 21 – Um Exemplo de Herança entre Classes

A representação gráfica UML de uma Generalização é uma forma de triângulo oco no final da superclasse da linha (ou árvore de linhas) que a conecta a um ou mais subtipos. O relacionamento de generalização também é conhecido como o relacionamento de herança ou "é um". A superclasse (classe base) no relacionamento de generalização também é conhecida como "pai", superclasse, classe base ou tipo base. O subtipo no relacionamento de especialização também é conhecido como "filho", subclasse, classe derivada, tipo derivado, classe herdada ou tipo herdado.

#ATENÇÃO#

Note que esta relação não tem nenhuma semelhança com a relação biológica entre pais e filhos: o uso desses termos é extremamente comum, mas pode ser enganoso.

#ATENÇÃO#

Por exemplo, "um pau-brasil é um tipo de árvore", "um automóvel é um tipo de veículo". A generalização só pode ser mostrada em diagramas de classes e em diagramas de casos de uso (PENDER, 2004).

Multiplicidade

Esse relacionamento de associação indica que (pelo menos) uma das duas classes relacionadas faz referência ao outro. Este relacionamento é geralmente descrito como "A tem um B" (uma mãe gato tem gatinhos, gatinhos têm um gato mãe). A representação UML de uma associação é uma linha que conecta as duas classes associadas. Em cada extremidade da linha há notação opcional. Por exemplo, podemos indicar, usando uma ponta de seta, que a extremidade pontiaguda é visível a partir da ponta da seta (PENDER, 2004). Podemos indicar a propriedade pela colocação de uma bola, o papel que os elementos desse fim desempenham fornecendo um nome para o papel e a multiplicidade de instâncias dessa entidade (o intervalo de número de objetos que participam da associação do ponto de vista da perspectiva do outro lado).

- 0 Nenhum caso (raro)
- 0..1 Nenhuma instância ou uma instância
- 1 Exatamente uma instância
- 1..1 Exatamente uma instância
- 0 .. * Zero ou mais instâncias
- Zero ou mais instâncias
- 1 .. * Um ou mais instâncias

Diagrama de Atividade

Os diagramas de atividades são representações gráficas de fluxos de trabalho de atividades e ações passo a passo com suporte para escolha, iteração e simultaneidade. Na UML, os diagramas de atividades destinam-se a modelar processos computacionais e organizacionais (ou seja, fluxos de trabalho), bem como os fluxos de dados que se cruzam com as atividades relacionadas. Embora os diagramas de atividades mostrem principalmente o fluxo geral de controle, eles também podem incluir elementos que mostram o fluxo de dados entre atividades por meio de um ou mais armazenamentos de dados (PENDER, 2004).

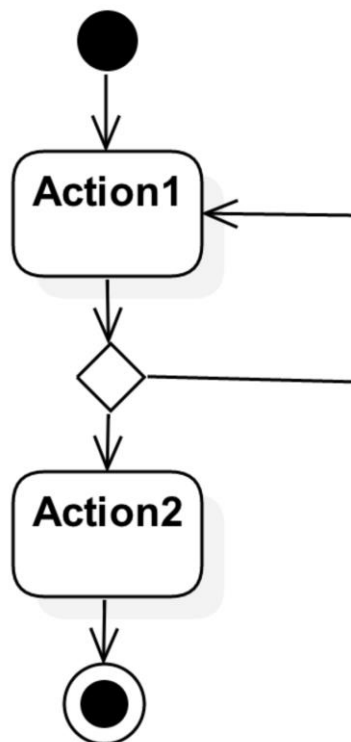


Figura 22 – Um Exemplo de Diagrama de Atividade

Construção

Os diagramas de atividades são construídos a partir de um número limitado de formas, conectadas com setas. Os tipos de formas mais importantes:

- Elipses representam ações
- Diamantes representam decisões
- Barras representam o início (divisão) ou final (união) de atividades simultâneas
- Um círculo preto representa o início (nó inicial) do fluxo de trabalho
- Um círculo preto cercado representa o final (nó final)
- As setas correm desde o início até o final e representam a ordem em que as atividades acontecem

Os diagramas de atividades podem ser considerados como uma forma de fluxograma estruturado combinado com um diagrama de fluxo de dados tradicional. As técnicas típicas de fluxograma não possuem construções para expressar a simultaneidade. No entanto, os símbolos de junção e divisão nos diagramas de atividade resolvem isso apenas para casos simples; o significado do modelo não é claro quando eles são combinados arbitrariamente com decisões ou loops (PENDER, 2004).

#REFLITA#

Como os diagramas de atividade podem auxiliar o analista a entender o problema que ele está modelando?

REFLITA

Enquanto na UML 1.x, os diagramas de atividades eram uma forma especializada de diagramas de estados, na UML 2.x, os diagramas de atividades foram reformuladas para serem baseados na semântica semelhante à rede de Petri, aumentando o escopo de situações que podem ser modeladas usando diagramas de atividades. Essas alterações fazem com que muitos diagramas de atividades da UML 1.x sejam interpretados de maneira diferente na UML 2.x (PENDER, 2004).

Os diagramas de atividade UML na versão 2.x podem ser usados em vários domínios, por exemplo, no design de sistemas embarcados. É possível verificar essa especificação usando a técnica de verificação de modelos.

Diagrama de Máquina de Estados

A máquina de estados UML, também conhecida como *statechart* UML, é uma realização significativamente aprimorada do conceito matemático de um autômato finito em aplicações de ciência da computação, conforme expresso na notação UML. Os conceitos por trás disso são sobre

como organizar um dispositivo, programa de computador ou outro processo (muitas vezes técnico) de tal forma que uma entidade ou cada uma de suas sub-entidades esteja sempre em exatamente um dentre vários estados possíveis e onde haja boas transições condicionais definidas entre esses estados (PENDER, 2004).

#REFLITA#

Os estados de uma entidade são muito importantes para definir onde inicia o ciclo de vida e onde termina. Todas entidades tem uma máquina de estado.

REFLITA

A máquina de estados UML é uma variante baseada em objetos do Harel *statechart*, adaptada e estendida pela UML. O objetivo das máquinas de estado UML é superar as principais limitações das máquinas tradicionais de estado finito, mantendo seus principais benefícios. Os *statecharts* da UML introduzem os novos conceitos de estados aninhados hierarquicamente e regiões ortogonais, enquanto ampliam a noção de ações. As máquinas de estado UML têm as características das máquinas *Mealy* e das máquinas Moore. Eles suportam ações que dependem tanto do estado do sistema quanto do evento desencadeador, como em máquinas *Mealy*, bem como ações de entrada e saída, associadas a estados e não a transições, como nas máquinas Moore (PENDER, 2004).

O termo "máquina de estados UML" pode se referir a dois tipos de máquinas de estado: máquinas de estado comportamental e máquinas de estado de protocolo. Máquinas de estado comportamental podem ser usadas para modelar o comportamento de entidades individuais (por exemplo, instâncias de classe). Máquinas de estado de protocolo são usadas para expressar protocolos de uso e podem ser usadas para especificar os cenários de uso legal de classificadores, interfaces e portas.

Conceitos básicos de máquina de estado

Muitos sistemas de software são orientados a eventos, o que significa que eles esperam continuamente pela ocorrência de algum evento externo ou interno, como um clique do mouse, um pressionamento de botão, um registro de tempo ou a chegada de um pacote de dados. Depois de reconhecer o evento, esses sistemas reagem executando a computação apropriada que pode incluir a manipulação do hardware ou a geração de eventos "*soft*" que acionam outros componentes de software internos. (Por isso, os sistemas acionados por eventos são alternativamente chamados de sistemas reativos.) Quando o tratamento de eventos é concluído, o sistema volta a esperar pelo próximo evento.

#REFLITA#

Eventos determinam estados. Pense nisso quando estiver criando suas máquinas pois eles mostram muito como os estados devem ser.

REFLITA

A resposta a um evento geralmente depende tanto do tipo do evento quanto do estado interno do sistema e pode incluir uma mudança de estado levando a uma transição de estado. O padrão de eventos, estados e transições de estado entre esses estados pode ser abstraído e representado como uma máquina de estado finito (FSM).

O conceito de um FSM é importante na programação orientada a eventos porque torna o tratamento de eventos explicitamente dependente tanto do tipo de evento quanto do estado do sistema. Quando usada corretamente, uma máquina de estado pode reduzir drasticamente o número de caminhos de execução através do código, simplificar as condições testadas em cada ponto de ramificação e simplificar a alternância entre diferentes modos de execução. Por outro lado, o uso de programação orientada a eventos sem um modelo FSM subjacente pode levar os programadores a produzir código de aplicativo propenso a erros, difícil de estender e excessivamente complexo (PENDER, 2004).

Diagramas básicos de estado UML

A UML preserva a forma geral dos diagramas de estado tradicionais. Os diagramas de estado da UML são gráficos direcionados nos quais os nós denotam estados e conectores denotam transições de estado. Por exemplo, a Figura 23 mostra um diagrama de estado UML correspondente à máquina de estado do teclado do computador. Na UML, os estados são representados como retângulos arredondados rotulados com nomes de estado.

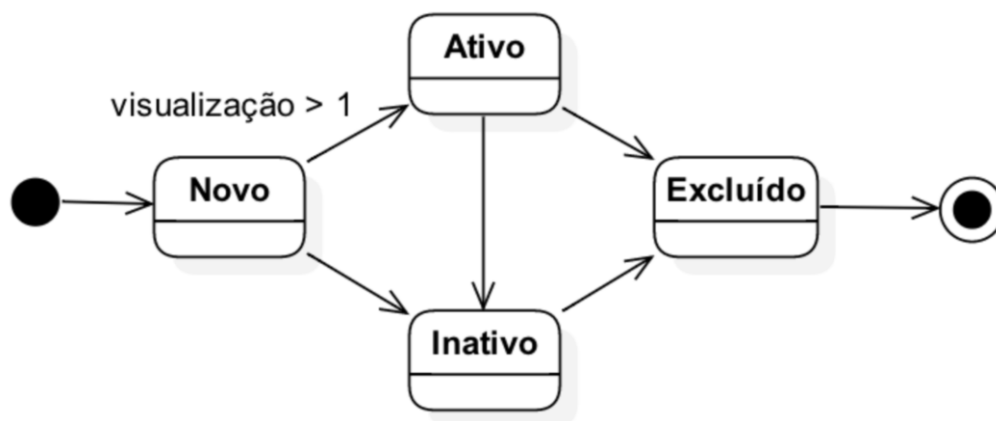


Figura 23 – Um Exemplo de Diagrama de Máquina de Estados

As transições, representadas como setas, são rotuladas com os eventos desencadeadores seguidos, opcionalmente, pela lista de ações executadas. A transição inicial se origina do círculo sólido e especifica o estado padrão quando o sistema é iniciado pela primeira vez. Cada diagrama de estado deve ter essa transição, que não deve ser rotulada, pois não é acionada por um evento. A transição inicial pode ter ações associadas.

Eventos

Um evento é algo que acontece e afeta o sistema. Estritamente falando, na especificação da UML, o termo evento refere-se ao tipo de ocorrência e não a qualquer ocorrência concreta dessa ocorrência. Por exemplo, ao sair do **novo** para **ativo** é um evento para a entidade, mas cada vez que a entidade é alterada não é um evento, desde que não altere o estado da mesma. Outro exemplo de evento de interesse para a entidade pode ser o **ativo -> inativo**, que poderia ser acionado pelo usuário ou por um temporizador (por exemplo, após 30 dias sem movimentos, fique inativo).

#REFLITA#

Existe um evento que envia uma entidade para o seu estado final. Esse estado final é conhecido também como estado absorvente pois a entidade não consegue mais sair desse estado.

REFLITA

Um evento pode ter parâmetros associados, permitindo que a instância do evento transmita não apenas a ocorrência de algum incidente interessante, mas também informações quantitativas sobre essa ocorrência. Por exemplo, o evento **ativo -> inativo** gerado por um temporizador tem parâmetros associados que no caso é a quantidade de dias de inatividade.

Uma instância de evento sobrevive à ocorrência instantânea que a gerou e pode transmitir essa ocorrência para uma ou mais máquinas de estado. Uma vez gerada, a instância do evento passa por um ciclo de vida de processamento que pode consistir em até três estágios. Primeiro, a instância do evento é recebida quando é aceita e aguarda o processamento (por exemplo, ela é colocada na fila de eventos). Posteriormente, a instância do evento é despachada para a máquina de estado, no ponto em que se torna o evento atual. Por fim, é consumido quando a máquina de estado termina o processamento da instância do evento. Uma instância de evento consumida não está mais disponível para processamento (PENDER, 2004).

Estados

Cada máquina de estado tem um estado que governa a reação da máquina de estado aos eventos. Um estado pode abstrair todas as sequências de eventos possíveis (mas irrelevantes) e capturar somente as relevantes (PENDER, 2004).

No contexto de máquinas de estado de software (e especialmente FSMs clássicas), o termo estado é frequentemente entendido como uma variável de estado único que pode assumir apenas um número limitado de valores determinados a priori (por exemplo, dois valores no caso do teclado ou mais geralmente - algum tipo de variável com um tipo enum em muitas linguagens de programação). A ideia de variável de estado (e modelo clássico de FSM) é que o valor da variável de estado define completamente o estado atual do sistema em qualquer momento. O conceito de estado reduz o problema de identificar o contexto de execução no código para testar apenas a variável de estado em vez de muitas variáveis, eliminando assim muita lógica condicional.

Estados estendidos

Na prática, entretanto, interpretar todo o estado da máquina de estado como uma variável de estado único rapidamente se torna impraticável para todas as máquinas de estado além das mais simples. De fato, mesmo se tivermos um único inteiro de 32 bits em nosso estado de máquina, ele poderá contribuir para mais de 4 bilhões de estados diferentes - e levará a uma explosão prematura do estado. Essa interpretação não é prática, portanto, em máquinas de estado UML, todo o estado da máquina de estado é geralmente dividido em (a) variável de estado enumerável e (b) todas as outras variáveis que são nomeadas como estado estendido. Outra maneira de ver isso é interpretar a variável de estado enumerável como um aspecto qualitativo e um estado estendido como aspectos quantitativos de todo o estado. Nessa interpretação, uma mudança de variável nem sempre implica uma mudança dos aspectos qualitativos do comportamento do sistema e, portanto, não leva a uma mudança de estado (PENDER, 2004).

Máquinas de estado suplementadas com variáveis de estado estendido são chamadas de máquinas de estado estendidas e as máquinas de estado UML pertencem a essa categoria. Máquinas de estado estendido podem aplicar o formalismo subjacente a problemas muito mais complexos do que é prático sem incluir variáveis de estado estendidas. Por exemplo, se tivermos que implementar algum tipo de limite em nosso modelo (digamos, limitar o número de **ativo** -> **inativo** para 10), sem estado estendido, precisaríamos criar e processar 10 estados - o que não é prático; no entanto, com uma máquina de estado estendida, podemos introduzir uma variável *key_count*, que é inicializada em 10 e decrementada por **ativo** -> **inativo** sem alterar a variável de estado.

O diagrama de estados da Figura 23 é um exemplo de uma máquina de estados estendida, na qual a condição completa do sistema (chamada de estado estendido) é a combinação de um aspecto qualitativo - a variável de estado - e os aspectos quantitativos - as variáveis de estado estendidas.

#REFLITA#

Pense nas vantagens de utilizar todos os elementos de uma máquina de estados do ponto de vista do desenvolvedor.

REFLITA

A vantagem óbvia das máquinas de estado estendido é a flexibilidade. Por exemplo, alterar o limite governado por *key_count* de 1000 a 10000 alterações **ativo** -> **inativo** de tecla não complicaria a máquina de estado estendida. A única modificação necessária seria alterar o valor de inicialização da variável de estado estendido *key_count* durante a inicialização.

Essa flexibilidade de máquinas de estado estendido tem um preço, no entanto, devido ao complexo acoplamento entre os aspectos "qualitativo" e "quantitativo" do estado estendido. O acoplamento ocorre através das condições de guarda anexadas às transições, conforme mostrado na Figura 23.

Condições de guarda

Condições de guarda são expressões booleanas avaliadas dinamicamente com base no valor de variáveis de estado estendidas e parâmetros de evento. As condições de proteção afetam o comportamento de uma máquina de estado, permitindo ações ou transições apenas quando elas são avaliadas como TRUE e desativadas quando avaliadas como FALSE. Na notação UML, as condições de guarda são mostradas entre colchetes (por exemplo, [*visualização* > 1] na Figura 23.

#REFLITA#

As condições de guarda estão associadas com regras de negócio. Pense em termos do ponto de vista do negócio ao defini-las.

REFLITA

A necessidade de guardas é a consequência imediata de adicionar variáveis de estado estendido de memória ao formalismo de máquina de estado. Usado com moderação, as variáveis de estado estendido e as proteções formam um mecanismo poderoso que pode simplificar os projetos. Por outro lado, é possível abusar facilmente de estados estendidos e guardas (PENDER, 2004).

Ações e transições

Quando uma instância de evento é despachada, a máquina de estado responde executando ações, como alterar uma variável, executar E/S, chamar uma função, gerar outra instância de evento ou alterar para outro estado. Quaisquer valores de parâmetro associados ao evento atual estão disponíveis para todas as ações causadas diretamente por esse evento. A mudança de um estado para outro é chamada de transição de estado, e o evento que a causa é chamado de evento desencadeador ou simplesmente o acionador (PENDER, 2004).

Em máquinas de estados estendidos, uma transição pode ter um protetor, o que significa que a transição pode "disparar" somente se a proteção for avaliada como TRUE. Um estado pode ter muitas transições em resposta ao mesmo acionador, desde que eles tenham proteções não sobrepostas; no entanto, essa situação pode criar problemas na sequência de avaliação das proteções quando o gatilho comum ocorre. A especificação UML intencionalmente não estipula nenhuma ordem específica; em vez disso, a UML coloca o ônus sobre o projetista de planejar guardas de tal forma que a ordem de sua avaliação não importe. Na prática, isso significa que as expressões de guarda não devem ter efeitos colaterais, pelo menos nenhuma que altere a avaliação de outros guardas que tenham o mesmo gatilho (PENDER, 2004).

Modelo Run-to-Completion (RTC)

Todos os formalismos de máquina de estado, incluindo máquinas de estado UML, assumem

universalmente que uma máquina de estado conclui o processamento de cada evento antes de poder iniciar o processamento do próximo evento. Esse modelo de execução é chamado de execução para conclusão ou RTC (run-to-completion) (PENDER, 2004).

No modelo RTC, o sistema processa eventos em etapas RTC discretas e indivisíveis. Novos eventos de entrada não podem interromper o processamento do evento atual e devem ser armazenados (normalmente em uma fila de eventos) até que a máquina de estado fique inativa novamente. Essas semânticas evitam completamente quaisquer problemas internos de simultaneidade em uma única máquina de estado. O modelo RTC também contorna o problema conceitual de processamento de ações associadas a transições, em que a máquina de estado não está em um estado bem definido (está entre dois estados) durante a duração da ação. Durante o processamento de eventos, o sistema não responde (não é observável), portanto, o estado mal definido durante esse período não tem significado prático.

Observe, no entanto, que o RTC não significa que uma máquina de estado tenha que monopolizar a CPU até que a etapa RTC esteja concluída. A restrição de preempção só se aplica ao contexto da tarefa da máquina de estado que já está ocupada processando eventos. Em um ambiente multitarefa, outras tarefas (não relacionadas ao contexto da tarefa da máquina de estado ocupado) podem estar em execução, possivelmente antecipando a máquina de estados em execução no momento. Enquanto outras máquinas de estado não compartilharem variáveis ou outros recursos entre si, não haverá riscos de simultaneidade.

A principal vantagem do processamento RTC é a simplicidade. Sua maior desvantagem é que a capacidade de resposta de uma máquina de estado é determinada por sua etapa RTC mais longa. A obtenção de etapas curtas de RTC pode complicar significativamente os projetos em tempo real (PENDER, 2004).

Considerações Finais

Neste estudo aprendemos sobre o que é um projeto OO e quais alguns dos diagramas que podemos utilizar para a construção do mesmo. Esses diagramas são os fundamentos da UML e representam a estrutura de dados, via diagrama de classes, a sequência de atividades, via diagrama de atividades e os estados possíveis que uma entidade pode assumir, via máquina de estados. Esses diagramas tem uma função importante para realizar um projeto OO permitindo ao desenvolvedor entender a solução do problema por alguns pontos de vista diferentes.

UNIDADE 4

TÓPICOS AVANÇADOS

Objetivos de aprendizagem

- Aprender sobre a separação de responsabilidades
- Aprender sobre o *Domain Driven Development*
- Aprender sobre BPMN, DMN e CMN

Introdução

Vimos o básico da construção de um projeto orientado a objetos. No entanto, existem diversas técnicas que podem ser exploradas para melhoria do resultado final, sendo uma análise de melhor qualidade ou um projeto de melhor qualidade. O importante é saber a técnica e como utilizá-la da melhor forma possível. Para isso, vamos estudar uma notação muito útil para a diagramação de processos chamada BPMN. Vamos estudar também a notação DMN que serve para a criação de regras baseadas em *IF-THEN*. E por fim, vamos estudar a notação CMN que serve para representação de casos. Tudo isso associado a separação de responsabilidades e o desenvolvimento orientado ao domínio.

Separação de Responsabilidades

Na ciência da computação, a separação de responsabilidades (SoC) é um princípio de design para separar um programa de computador em seções distintas, de modo que cada seção aborda uma preocupação separada. Uma preocupação é um conjunto de informações que afeta o código de um programa de computador. Uma preocupação pode ser tão geral quanto os detalhes do hardware para o qual o código está sendo otimizado, ou tão específico quanto o nome de uma classe para instanciar. Um programa que incorpora SoC bem é chamado de programa modular (EVANS, 2003). A modularidade e, portanto, a separação de interesses, é obtida encapsulando informações dentro de uma seção de código que possui uma interface bem definida. O encapsulamento é um meio de esconder informação. Projetos em camadas em sistemas de informação são outra forma de realização da separação de interesses (por exemplo, camada de apresentação, camada de lógica de negócios, camada de acesso a dados, camada de persistência).

A separação de interesses resulta em graus mais altos de liberdade em algum aspecto do projeto, implantação ou uso do programa. Comum entre estes é maior grau de liberdade para simplificação e manutenção de código. Quando as preocupações são bem separadas, há graus mais altos de liberdade para reutilização de módulos, bem como desenvolvimento e atualização independentes. Como os módulos ocultam os detalhes de suas preocupações por trás das interfaces, aumentam os resultados de liberdade para melhorar ou modificar posteriormente a seção de código de uma única preocupação sem precisar conhecer os detalhes de outras seções e sem precisar fazer alterações correspondentes nessas seções. Os módulos também podem expor diferentes versões de uma interface, o que aumenta a liberdade de atualizar um sistema complexo

de maneira gradativa, sem perda temporária de funcionalidade (EVANS, 2003).

Separação de preocupações é uma forma de abstração. Como na maioria das abstrações, as interfaces devem ser adicionadas e geralmente há mais código de rede a ser executado. Portanto, apesar dos muitos benefícios de preocupações bem separadas, muitas vezes há uma penalidade de execução associada.

Implementação

Os mecanismos para programação modular ou orientada a objetos que são fornecidos por uma linguagem de programação são mecanismos que permitem aos desenvolvedores fornecer SoC. Por exemplo, linguagens de programação orientadas a objeto, como C #, C ++, Delphi e Java podem separar preocupações em objetos, e padrões de design de arquitetura como MVC ou MVP podem separar conteúdo de apresentação e processamento de dados (modelo) de conteúdo. O design orientado a serviços pode separar as preocupações em serviços. Linguagens procedurais de programação, como C e Pascal, podem separar preocupações em procedimentos ou funções. Linguagens de programação orientadas a aspectos podem separar preocupações em aspectos e objetos (EVANS, 2003).

Separação de preocupações é um princípio de design importante em muitas outras áreas, bem como, planejamento urbano, arquitetura e design de informação. O objetivo é compreender, projetar e gerenciar com eficiência sistemas complexos interdependentes, para que as funções possam ser reutilizadas, otimizadas independentemente de outras funções e isoladas da possível falha de outras funções.

Exemplos comuns incluem separar um espaço em salas, de modo que a atividade em um cômodo não afete pessoas em outros cômodos, e manter o fogão em um circuito e as luzes em outro, de modo que a sobrecarga do fogão não apague as luzes. O exemplo com salas mostra o encapsulamento, em que as informações dentro de um cômodo, como a bagunça, não estão disponíveis para os outros cômodos, exceto pela interface, que é a porta. O exemplo com circuitos demonstra que a atividade dentro de um módulo, que é um circuito com consumidores de eletricidade conectados, não afeta a atividade em um módulo diferente, então cada módulo não está preocupado com o que acontece no outro (EVANS, 2003).

Origem

O termo separação de preocupações provavelmente foi cunhado por Edsger W. Dijkstra em seu artigo de 1974 "Sobre o papel do pensamento científico". Sabemos que um programa deve estar correto e podemos estudá-lo apenas desse ponto de vista; Também sabemos que deve ser eficiente e podemos estudar sua eficiência em outro dia, por assim dizer. Em outro estado de espírito, podemos nos perguntar se, e em caso afirmativo: por que, o programa é desejável. Mas nada é ganho - pelo contrário! - abordando esses vários aspectos simultaneamente. É o que eu às vezes chamei de "separação das preocupações", a qual, mesmo que não seja perfeitamente possível, é a única técnica disponível para ordenar efetivamente os pensamentos de alguém, que eu conheço. É isso que quero dizer com "concentrar a atenção em algum aspecto": não significa ignorar os outros aspectos, é apenas fazer justiça ao fato de que, do ponto de vista desse aspecto, o

outro é irrelevante. Está sendo uma e múltiplas pistas ocupadas simultaneamente.

Idealmente, o programador deve ser capaz de se concentrar na primeira das três tarefas (descrevendo o que deve ser calculado) sem se distrair com as outras duas tarefas, mais administrativas. Claramente, a administração é importante, mas ao separá-la da tarefa principal, é provável que obtenhamos resultados mais confiáveis e podemos facilitar o problema de programação, automatizando grande parte da administração.

A separação de interesses também tem outras vantagens. Por exemplo, a comprovação de programas torna-se muito mais viável quando detalhes do sequenciamento e gerenciamento de memória estão ausentes do programa. Além disso, as descrições do que deve ser computado devem estar livres de descrições detalhadas, passo a passo, de como fazê-lo, para serem avaliadas com arquiteturas de máquinas diferentes. Sequências de pequenas alterações em um objeto de dados mantidos em uma loja podem ser uma descrição inadequada de como calcular algo quando uma máquina altamente paralela está sendo usada com milhares de processadores distribuídos pela máquina e instalações de armazenamento locais em vez de globais. Automatizar os aspectos administrativos significa que o implementador de linguagem tem que lidar com eles, mas ele tem muito mais oportunidade de fazer uso de mecanismos de computação muito diferentes com arquiteturas de máquinas diferentes.

Exemplos

Pilha de protocolos da Internet

A separação de interesses é crucial para o design da Internet. No *Internet Protocol Suite*, grandes esforços foram feitos para separar as preocupações em layouts bem definidos. Isso permite que os designers de protocolo se concentrem nas preocupações de uma camada e ignorem as outras camadas. O protocolo SMTP da camada de aplicação, por exemplo, está preocupado com todos os detalhes da condução de uma sessão de e-mail em um serviço de transporte confiável (geralmente TCP), mas não no mínimo preocupado sobre como o serviço de transporte torna esse serviço confiável. Da mesma forma, o TCP não está preocupado com o roteamento de pacotes de dados, que é tratado na Camada da Internet.

HTML, CSS, JavaScript

Linguagem de marcação de hipertexto (HTML), folhas de estilos em cascata (CSS) e JavaScript (JS) são linguagens complementares usadas no desenvolvimento de páginas da Web e sites. O HTML é usado principalmente para organização de conteúdo de páginas da Web, CSS é usado para definição de estilo de apresentação de conteúdo e JS define como o conteúdo interage e se comporta com o usuário. Historicamente, esse não era o caso: antes da introdução do CSS, o HTML desempenhava ambos os deveres de definir semântica e estilo.

Programação orientada por aspecto

A programação orientada ao assunto permite que preocupações separadas sejam abordadas como construções de software separadas, cada uma em pé de igualdade com as outras. Cada preocupação fornece sua própria estrutura de classe na qual os objetos em comum são organizados e contribui com o estado e os métodos para o resultado composto em que eles se cruzam. As regras de correspondência descrevem como as classes e os métodos nas várias preocupações estão relacionados entre si nos pontos em que eles interagem, permitindo que o comportamento composto de um método seja derivado de várias preocupações. A Separação de Preocupações Multi-Dimensional permite que a análise e a composição das preocupações sejam manipuladas como uma "matriz" multidimensional na qual cada preocupação fornece uma dimensão na qual diferentes pontos de escolha são enumerados, com as células da matriz ocupadas pelo apropriado. artefatos de software.

Programação orientada a aspectos

A programação orientada a aspectos permite que as preocupações transversais sejam abordadas como preocupações principais. Por exemplo, a maioria dos programas requer alguma forma de segurança e registro. A segurança e o registro em log geralmente são preocupações secundárias, enquanto a principal preocupação é com frequência atingir metas de negócios. No entanto, ao projetar um programa, sua segurança deve ser incorporada ao projeto desde o início, em vez de ser tratada como uma preocupação secundária. A aplicação da segurança posteriormente resulta em um modelo de segurança insuficiente que deixa muitas lacunas para ataques futuros. Isso pode ser resolvido com programação orientada a aspectos. Por exemplo, um aspecto pode ser escrito para impor que as chamadas para uma determinada API sejam sempre registradas ou que os erros sempre sejam registrados quando uma exceção é lançada, independentemente de o código procedural do programa manipular a exceção ou propagá-la.

Automação de construção de software

A maioria das tarefas de organização do projeto são vistas como tarefas secundárias. Por exemplo, a automação de construção é uma abordagem para automatizar o processo de compilação do código-fonte em código binário. Os principais objetivos da automação de construção são reduzir o risco de erro humano e economizar tempo.

Domain Driven Development (DDD)

O design orientado por domínio (DDD) é uma abordagem para o desenvolvimento de software para necessidades complexas, conectando a implementação a um modelo em evolução. A premissa do design orientado a domínio é o seguinte:

- Colocando o foco principal do projeto no domínio principal e na lógica do domínio;
- Baseando desenhos complexos em um modelo do domínio;
- Iniciar uma colaboração criativa entre especialistas técnicos e de domínio para refinar, de forma iterativa, um modelo conceitual que aborde problemas de domínio específicos.

Conceitos

Ao criar um modelo sempre lançamos mão de alguns conceitos. Neste caso, alguns deles são conhecidos e utilizados com o DDD. Dessa forma, conceitos do modelo incluem:

- Contexto: O cenário em que aparece uma palavra ou declaração que determina seu significado;
- Domínio: Uma esfera de conhecimento (ontologia), influência ou atividade. A área de assunto para a qual o usuário aplica um programa é o domínio do software;
- Modelo: Um sistema de abstrações que descreve aspectos selecionados de um domínio e pode ser usado para resolver problemas relacionados a esse domínio;
- Linguagem Ubíqua: Uma linguagem estruturada em torno do modelo de domínio e usada por todos os membros da equipe para conectar todas as atividades da equipe ao software.

Design dirigido por domínio estratégico

Rede semântica de padrões no design dirigido por domínio estratégico. Idealmente, seria preferível ter um modelo único e unificado. Embora esse seja um objetivo nobre, na realidade, ele geralmente se fragmenta em vários modelos. É útil reconhecer esse fato da vida e trabalhar com ele. O Design Estratégico é um conjunto de princípios para manter a integridade do modelo, destilação do Modelo de Domínio e trabalhar com vários modelos.

Contexto Limitado

Vários modelos estão em jogo em qualquer projeto grande. No entanto, quando o código baseado em modelos distintos é combinado, o software se torna defeituoso, pouco confiável e difícil de entender. A comunicação entre os membros da equipe se torna confusa. Muitas vezes não está claro em que contexto um modelo não deve ser aplicado. Portanto: Defina explicitamente o contexto no qual um modelo se aplica. Explicitamente estabeleça limites em termos de organização da equipe, uso em partes específicas do aplicativo e manifestações físicas, como bases de código e esquemas de banco de dados. Mantenha o modelo estritamente consistente dentro desses limites, mas não se distraia ou confunda com problemas externos.

Integração Contínua

Quando várias pessoas estão trabalhando no mesmo contexto limitado, há uma forte tendência para o modelo se fragmentar. Quanto maior a equipe, maior o problema, mas apenas três ou quatro pessoas podem encontrar sérios problemas. No entanto, dividir o sistema em contextos cada vez menores acaba perdendo um valioso nível de integração e coerência. Portanto: Instale um processo de mesclagem de todos os códigos e outros artefatos de implementação com frequência, com testes automatizados para sinalizar a fragmentação rapidamente. Exercer incansavelmente a linguagem onipresente para elaborar uma visão compartilhada do modelo à medida que os conceitos evoluem na cabeça de diferentes pessoas.

Mapa de Contexto

Um contexto limitado individual deixa alguns problemas na ausência de uma visão global. O contexto de outros modelos ainda pode ser vago e em fluxo. As pessoas de outras equipes não estarão muito conscientes dos limites do contexto e, sem saber, farão alterações que ofusquem as bordas ou compelissem as interconexões. Quando as conexões devem ser feitas entre diferentes contextos, elas tendem a sangrar umas nas outras. Portanto: Identifique cada modelo em jogo no projeto e defina seu contexto limitado. Isso inclui os modelos implícitos de subsistemas não orientados a objetos. Nomeie cada contexto delimitado e torne os nomes parte da linguagem onipresente. Descreva os pontos de contato entre os modelos, delineando a tradução explícita para qualquer comunicação e destacando qualquer compartilhamento. Mapeie o terreno existente.

Desvantagens

Para ajudar a manter o modelo como uma construção de linguagem pura e útil, a equipe deve normalmente implementar uma grande quantidade de isolamento e encapsulamento no modelo de domínio. Consequentemente, um sistema baseado em design orientado a domínio pode ter um custo relativamente alto. Embora o design orientado a domínio forneça vários benefícios técnicos, como manutenção, a Microsoft recomenda que ele seja aplicado apenas a domínios complexos em que o modelo e os processos linguísticos fornecem benefícios claros na comunicação de informações complexas e na formulação de um entendimento comum de o domínio.

BPMN, DMN e CMMN

BPMN

Exemplo de um modelo de processo de negócios e notação para um processo com um fluxo normal. O *Business Process Model e Notation* (BPMN) é uma representação gráfica para especificar processos de negócios em um diagrama de processo. A Figura 24 a seguir exibe um exemplo de

BPMN.

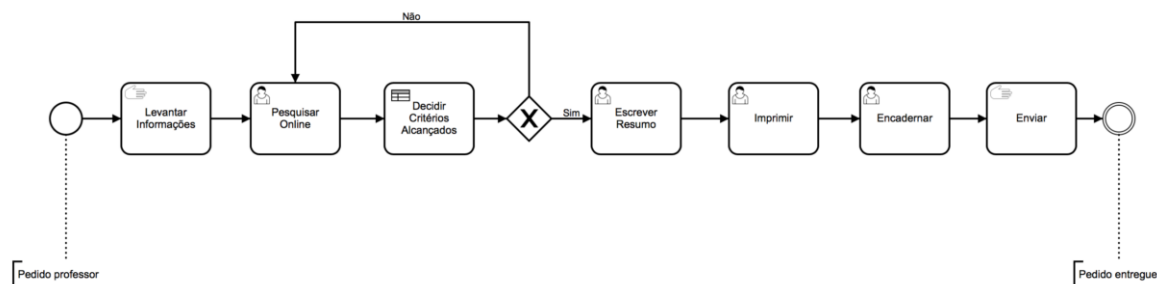


Figura 24 – Um Exemplo de Diagrama BPMN

A *Business Process Management Initiative* (BPMI) desenvolveu o BPMN, que foi mantido pelo *Object Management Group* (OMG) desde que as duas organizações se fundiram em 2005. A versão 2.0 do BPMN foi lançada em janeiro de 2011, ponto em que o nome foi adaptado para *Business Process Model and Notation* como semântica de execução também foram introduzidos ao lado dos elementos notacionais e de diagramação. Embora seja uma especificação OMG, o BPMN também é ratificado como ISO 19510. A última versão é BPMN 2.0.2, publicada em janeiro de 2014.

O *Business Process Model and Notation* (BPMN) é um padrão para modelagem de processos de negócios que fornece uma notação gráfica para especificar processos de negócios em um *Business Process Diagram* (BPD), baseado em uma técnica de fluxograma muito semelhante aos diagramas de atividades do *Unified Modeling Language* (UML). O objetivo do BPMN é oferecer suporte ao gerenciamento de processos de negócios, tanto para usuários técnicos quanto para usuários corporativos, fornecendo uma notação intuitiva para os usuários corporativos, capaz de representar uma semântica complexa de processos. A especificação BPMN também fornece um mapeamento entre os gráficos da notação e as construções subjacentes de linguagens de execução, particularmente BPEL (*Business Process Execution Language*) (IIBA, 2015).

O principal objetivo do BPMN é fornecer uma notação padrão facilmente compreensível por todos os participantes do negócio. Estes incluem os analistas de negócios que criam e refinam os processos, os desenvolvedores técnicos responsáveis por implementá-los e os gerentes de negócios que os monitoram e gerenciam. Consequentemente, o BPMN serve como uma linguagem comum, colmatando a lacuna de comunicação que frequentemente ocorre entre o design e a implementação do processo de negócios.

Atualmente, existem vários padrões concorrentes para linguagens de modelagem de processos de negócios usados por ferramentas e processos de modelagem. A adoção generalizada de um padrão único ajudaria a unificar a expressão de conceitos básicos de processos de negócios (por exemplo, processos públicos e privados, coreografias), bem como conceitos de processos avançados (por exemplo, manipulação de exceções, compensação de transações).

O BPMN foi complementado por dois novos padrões para a construção de modelos de gestão de casos e modelos de decisão, o *Case Management Model and Notation* (2013) e o *Decision Model and Notation* (2014).

Elementos

Os modelos BPMN são expressos por diagramas simples construídos a partir de um conjunto limitado de elementos gráficos. Tanto para usuários corporativos quanto para desenvolvedores, eles simplificam o entendimento do fluxo e do processo das atividades de negócios. As quatro categorias básicas de elementos do BPMN são:

- Objetos de fluxo
 - Eventos, atividades, gateways
- Conectando Objetos
 - Fluxo de sequência, fluxo de mensagens, associação
- Pistas de Natação
 - Piscina, pista
- Artefatos
 - Objeto de dados, grupo, anotação

Essas quatro categorias permitem a criação de diagramas de processos de negócios simples (BPDs). Os BPDs também permitem fazer novos tipos de objeto de fluxo ou artefato, para tornar o diagrama mais compreensível.

DMN

O *Decision Model and Notation* (DMN) é um padrão publicado pelo *Object Management Group* (OMG). É uma abordagem padrão para descrever e modelar decisões repetitivas dentro das organizações para garantir que os modelos de decisão sejam intercambiáveis entre as organizações. O padrão DMN fornece ao setor uma notação de modelagem para decisões que suportarão o gerenciamento de decisões e as regras de negócios. A notação é projetada para ser legível por usuários de negócios e de TI. Isso permite que vários grupos colaborem efetivamente na definição de um modelo de decisão:

- Os empresários que gerenciam e monitoram as decisões
- Os analistas de negócios ou analistas funcionais que documentam os requisitos de decisão inicial e especificam os modelos de decisão detalhados e a lógica de decisão
- Os desenvolvedores técnicos responsáveis pela automação dos sistemas que tomam as decisões

O padrão DMN pode ser efetivamente usado de forma independente, mas também é complementar aos padrões BPMN e CMMN. O BPMN define um tipo especial de atividade, a Tarefa de Regra de Negócios, que "fornece um mecanismo para o processo fornecer entrada a um mecanismo de regras de negócios e obter a saída de cálculos que o mecanismo de regras de negócios pode fornecer" que pode ser usado para mostrar onde em um processo BPMN uma

decisão definida usando DMN deve ser usada (IIBA, 2015).

Dish

[Show details](#)

U	Input +	Output +	Annotation
	Season	Dish	
1	"Fall"	"Spareribs"	-
2	"Winter"	"Roastbeef"	-
3	"Spring"	"Steak"	-
4	"Summer"	"Light Salad and a nice Steak"	Hey, why not!?
+	-	-	-

Figura 25 – Um Exemplo de Diagrama DMN

Diagramas de requisitos de decisão

Se você quiser discutir e analisar decisões complexas, que podem ser compostas de outras decisões, os diagramas de requisitos de decisão (DRD) podem ser úteis. Esta é uma notação bastante simples definida no padrão DMN, que basicamente consiste em

- Decisões: O ato de determinar um valor de saída a partir de um número de valores de entrada, usando a definição lógica. Essa lógica de decisão é o que você pode expressar nas tabelas de decisão.
- Dados de Entrada: Os dados de entrada que você "alimenta" em sua lógica de decisão para determinar o valor de saída.
- Relações entre Decisões: Você pode conectar decisões com setas e, portanto, indicar qual saída de decisão será considerada como uma entrada para outra decisão.
- Existem mais alguns símbolos na notação DRD, no entanto os mais relevantes são estes três. Devemos olhar um exemplo:

Esses elementos acima mostram tudo o que é necessário para a tomada de uma decisão. Além disso, ele mostra o relacionamento entre esses elementos o que torna esse diagrama interessante e útil para modelagem de decisões. A Figura 26 abaixo exibe um exemplo desse diagrama.

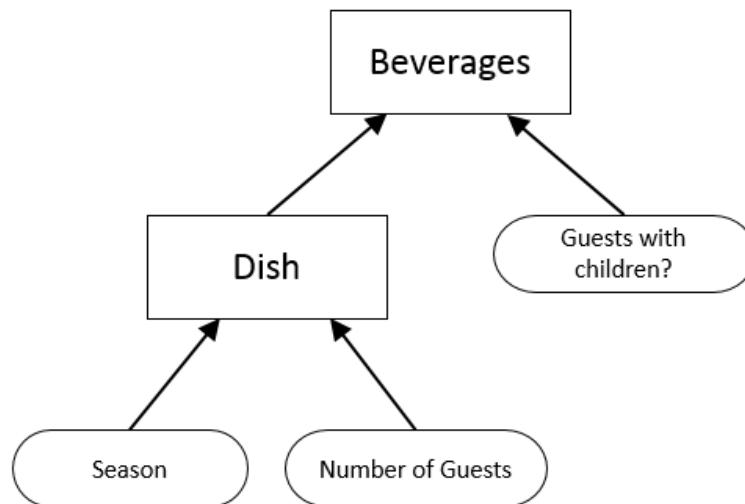


Figura 26 – Um Exemplo de Diagrama de Requisitos de Decisão

Benefícios

A utilização do padrão DMN melhorará a análise de negócios e o gerenciamento de processos de negócios:

- Outras técnicas populares de gerenciamento de requisitos, como BPMN e UML, não lidam com a tomada de decisão crescimento de projetos usando sistemas de gerenciamento de regras de negócio ou BRMS, que permitem mudanças mais rápidas
- Facilita a comunicação entre funções de negócios, TI e analíticas em uma empresa
- Fornece uma abordagem de modelagem de requisitos eficaz para projetos do *Predictive Analytics* e atende à necessidade de "entendimento do negócio" em metodologias para análise avançada, como o CRISP-DM.
- Fornece uma notação padrão para tabelas de decisão, o estilo mais comum de regras de negócios em um BRMS

CMMN

O *Case Management Model and Notation* (CMMN) é uma representação gráfica para expressar graficamente um caso, bem como um formato de troca para trocar modelos de Caso entre diferentes ferramentas. Um consórcio de 11 empresas contribuiu para o desenvolvimento do CMMN, que agora está sendo mantido pelo Grupo de Gerenciamento de Objetos. A versão 1.0 do CMMN foi lançada em maio de 2014 e alterada pela versão 1.1 em dezembro de 2016. A Figura 27 a seguir exibe um exemplo de CMMN.

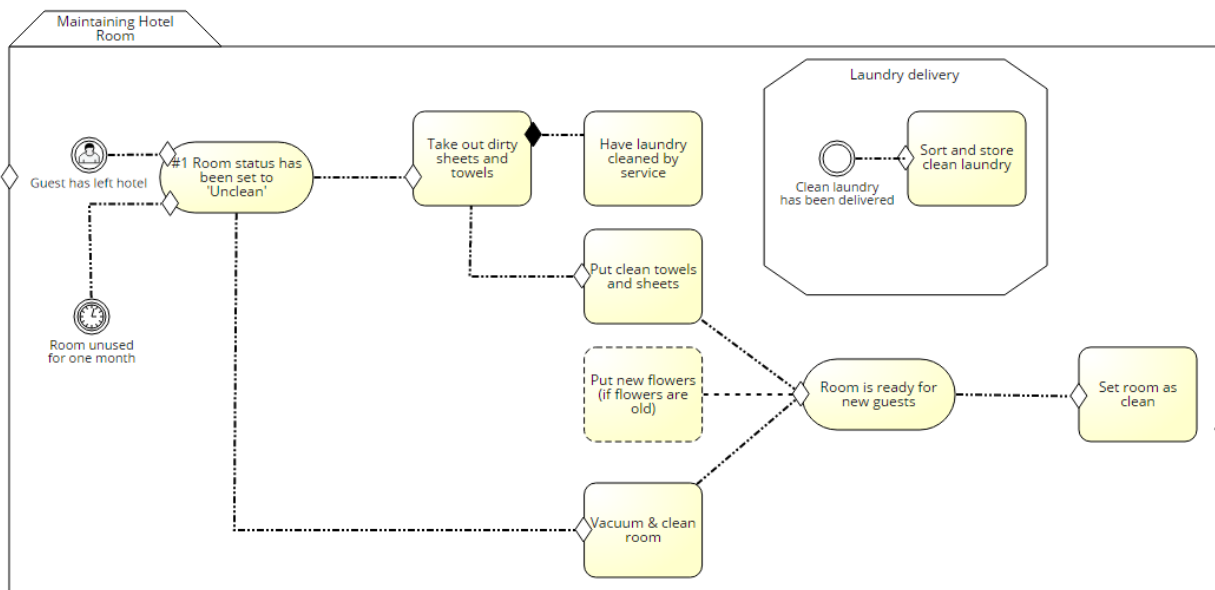


Figura 27 – Um Exemplo de Diagrama de CMMN

Em geral, muitos cenários de BPM incluem ações que podem divergir do fluxo de sequência comum, dentro de uma estrutura de tarefas definidas. O CMMN suporta a flexibilidade desses fluxos de trabalho. A notação é projetada para cenários em que um responsável pelo caso pode decidir em que ordem tarefas ou fluxos de sequência devem ser executados. Um fluxo de sequência CMMN pode ser acionado por um evento (ouvinte de evento), um estado (marco) ou com uma ação (tarefa).

Considerações Finais

Neste estudo aprendemos sobre o que vem a ser a separação de responsabilidade e como sua utilização pode melhorar a qualidade do software a ser produzido. Estudamos também as questões relacionadas com o DDD e sua aplicação no desenvolvimento de software. Esse assunto tem uma área de pesquisa aquecida e desperta o interesse de diversos pesquisadores ao redor do mundo. Por fim, vimos as notações BPMN, DMN e CMMN que auxiliam na criação de análise orientada a objeto suplementando a UML e deixando o resultado muito mais interessante.

UNIDADE 5

CENÁRIOS DE UTILIZAÇÃO

Objetivos de aprendizagem

- Cenários comuns de utilização de análise e projetos orientado a objetos
- Um caminho para análise e projeto
- Boas práticas de desenho

Introdução

Depois de entender os aspectos de análise e projeto orientado a objetos, podemos lançar mão agora a forma como utilizar isso de forma prática. Muitas vezes ficamos confusos sobre quando aplicar certas técnicas ou certos diagramas para resolver um determinado problema. Geralmente, essas dúvidas não são abordadas pela literatura. Aqui, vamos entender quais são os cenários mais comuns e como utilizar as técnicas que aprendemos anteriormente.

Cenários comuns de utilização de análise e projetos orientado a objetos

Vamos iniciar pela análise. O primeiro cenário mais comum é a forma como os requisitos são desenvolvidos. Esse é o ponto fundamental da análise. Como vimos anteriormente, o caso de uso é a técnica principal para isso. Obviamente que ao inicio do projeto não teremos informação suficiente para criar um diagrama completo de um caso de uso e muito menos seu descritivo. Então, como proceder com isso? Bom uma abordagem prática para resolver essa questão é aplicar o modelo iterativo e incremental. Com isso vamos desenvolvendo diferentes versões do diagrama de caso de uso e de seus descritivo sendo que a cada nova versão você terá uma versão mais completa e mais precisa que a versão anterior.

Vamos entender a sequencia de passos para criar a primeira versão e como evoluir a mesma para as demais.

1. Obtenha os atores envolvidos: isso é importante para entender quem irá trabalhar com o software a ser criado. Isso ajuda a entender quem executa determinadas atividades e qual o seu relacionamento com elas. Além disso, você pode utilizar esses atores para criar a hierarquia entre eles. Por exemplo, um analista de compras pode responder a um supervisor de compras. Essa técnica auxilia na criação dos perfis de acesso.
2. Obtenha os casos de uso: depois de entender quem são os atores, podemos agora entender o que cada um faz. Dessa forma, teremos o diagrama de casos de uso de uma maneira global. Isso permite que módulos sejam criados pelo agrupamento de funcionalidades, permite identificar se existe sobreposição de funções verificando se

existe mais de um ator executando mais de uma funcionalidade. Existe também a possibilidade de identificar se certas funcionalidades podem ser fatoradas (ou seja, reduzidas para um comportamento único) via generalização ou componentização.

3. Escrita dos diagramas de caso de uso: com as funcionalidades mapeadas via diagrama de caso de uso, podemos agora descrevê-las. Para isso, utilizamos o descritivo de caso de uso identificando as pré e pós-condições, fluxo principal, alternativos e de exceção. Identificamos também as regras de negócio.
4. Escrita das regras de negócio: separe as regras de negócio e as escreva na forma IF-THEN para que as mesmas possam ser testadas em tempo de análise.
5. Verificação: verifique se os artefatos produzidos são consistentes e estão livres de erros do ponto de vista de escrita.
6. Validação: valide se o que foi desenvolvido é o que precisa ser implementado. Isso pode ser feito com os usuários e clientes de diversas formas. A melhor delas é com um workshop sobre cada caso de uso.
7. Repita esses passos o quanto for necessário.

Uma outra questão que surge nesse momento é: quando meus casos de uso estão prontos para serem implementados? Essa resposta nem sempre é simples de ser obtida pois depende de vários fatores. O primeiro deles é o seu tempo disponível. Nem sempre é possível ficar escovando os casos de uso até que os mesmos fiquem perfeitos. Outro fator é o seu orçamento que na maior parte das vezes é restrito. Além disso, muitas vezes não temos a garantia de que a validação foi feita de maneira adequadas pois os usuários nem sempre sabem o que precisam. Vão descobrir ao utilizar o software criado.

#ATENÇÃO#

Quanto mais cedo você validar o que está sendo desenvolvido, menor será o custo de alteração. Códigos escritos e com dados produzidos geralmente apresentam um custo maior de manutenção em relação aos casos de uso.

#ATENÇÃO#

Depois dos casos de uso desenvolvidos podemos agora tratar dos aspectos de projeto. Em outras palavras, vamos desenhar uma possível solução para as funcionalidades que foram levantadas com os casos de uso. Da mesma forma, podemos ter uma sequencia de passos que auxilie nessa tarefa.

1. Identifique as entidades: com os casos de uso criados, podemos identificar as entidades que vamos utilizar para armazenar os dados e comportamentos. Para isso, podemos utilizar o diagrama de classes. Ele permite entender quais são as entidades (mapeadas por classes), quais as características (mapeadas por atributos) e quais seus comportamentos (mapeados por métodos). Além disso, esse diagrama permite

entender o relacionamento entre cada uma dessas entidades.

2. Identifique o ciclo de vida das entidades: com as entidades mapeadas podemos agora entender o ciclo de vida de cada uma. Isso pode ser feito via um diagrama de máquina de estados. Essa máquina de estado representa como cada entidade é criada, quais estados pode assumir e seu estado final.
3. Segmente as regras: verifique como segmentar as regras que foram mapeadas entre as classes criadas utilizando os métodos já identificados. Esse passo é muito importante e melhora muita a qualidade do software desenvolvido.
4. Verificação: verifique se os diagramas são consistentes (classes e estado) e procure por eventuais problemas de escrita.
5. Validação: valide com os desenvolvedores para verificar se essa solução é viável para ser implementada com a tecnologia escolhida pela equipe.
6. Repita esses passos o quanto for necessário.

Uma das questões mais importantes nesse contexto é entender como separar as regras. Isso é chamado de Particionamento de Regras e gera os mais acalorados debates sobre o tema. Em resumo, particionamento de regras quer dizer algo muito simples. Onde colocar as regras de negócio? Nos métodos das entidades? Nas classes auxiliares? Na interface de usuário? Nas rotinas de banco de dados? Em um middleware? Isso é uma questão de projeto, mais especificamente de arquitetura. Essa pergunta tem uma resposta padrão: depende. Onde colocar as regras de negócio sempre depende do projeto em questão. Mas podemos ter dois princípios básicos que se aplicam a todos os projetos:

- Duplicidade de código: independente de onde a regra for colocada, não deve existir duplicidade de código. Se isso acontecer a manutenção ficará prejudicada pois ao alterar em um local pode-se esquecer de alterar em outro.
- Testes: as regras devem ser bem testadas e dessa forma, se não forem bem separadas os testes ficarão prejudicados.

Um caminho para análise e projeto

Na seção anterior aprendemos os cenários e algumas técnicas para análise e projeto. Agora, de forma geral, como podemos partir do zero e chegar em um projeto pronto para ser implementado? Existem algumas abordagens para isso, como por exemplo o *Rational Unified Process* (RUP). Em linhas gerais, podemos extrair os fundamentos dessas abordagens. A Figura 28 a seguir representa isso.

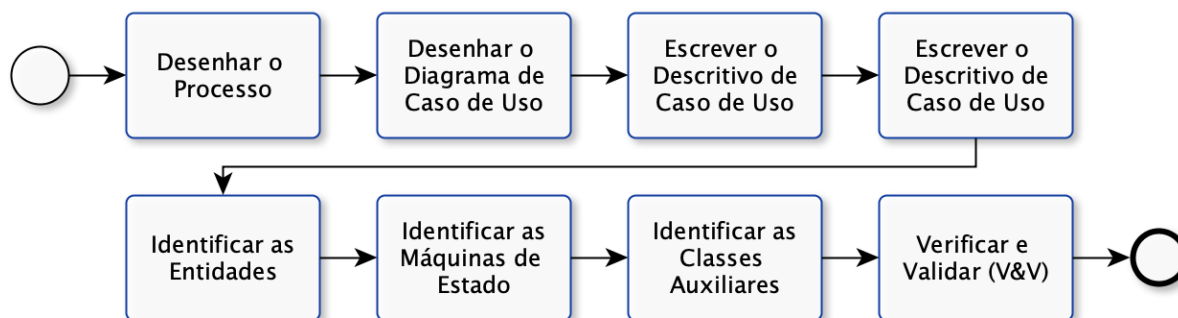


Figura 28 – Um Caminho para Análise e Projetos Orientados a Objetos

O início de tudo deve ter como base o desenho do processo ao qual o software se propõe suportar. Isso é muito importante e em algumas organizações existe uma área com um pessoal chamado de analista de negócios que tem como objetivo entender esses processos.

Apesar de a etapa de verificação e validação estar apresentada ao final do processo, sempre é interessante realizar isso a cada etapa. Se isso for feito, aumentam as chances de seu projeto estar mais adequado e próximo da realidade. Claro que isso representa um aumento dos custos também.

#ATENÇÃO#

Esse processo tem como finalidade ser um dos possíveis caminhos para criar uma análise e projeto orientado a objeto. Claro que o mesmo pode e deve ser adaptado ao seu processo.

#ATENÇÃO#

Boas Práticas de Projetos

Ao projetar software precisamos saber como criar as classes e principalmente como as mesmas se relacionam entre si. Dessa forma, a utilização de padrões podem ajudar nessa tarefa. Além disso, podemos lançar mão dos padrões de projeto para criar uma estrutura que possa acomodar facilmente modificações que possam (em sempre vão) surgir e se não bem projetado, haverá sempre muito esforço para modificar o software evitando os efeitos colaterais.

Além disso, existem padrões para a análise. A ideia é utilizar soluções para contextos que se repetem sempre na fase de análise e que possam ser replicados com um menor custo e maior qualidade. Geralmente, os padrões de projeto são mais conhecidos, mas os de análise não. Padrões são análise que foram úteis em algum contexto e devem permanecer úteis em outros. Por exemplo: certos padrões no domínio financeiro se aplicam a outros domínios também. Esses padrões apoiam o reuso de ideias durante a fase de análise. Existem padrões de análise em praticamente qualquer domínio. Martin Fowler escreveu um excelente livro que cobre padrões classificados como segue (FOWLER, 1996):

- Padrões de *Accountability* (organização e responsabilidade)
- Padrões de Observações e Medições
- Padrões de Observações para a Finança Corporativa
- Padrões de Inventário e Contabilidade
- Padrões de Planejamento
- Padrões para o Comércio
- Padrões de Contratos de Derivativos

Esses padrões auxiliam para melhorar a qualidade geral do software e possuem um baixo custo e risco de implementação.

#ATENÇÃO#

Entenda bem como utilizar os padrões para evitar a febre dos padrões. Isso em outras palavras que dizer que você fica colocando padrões em lugares que não são necessários.

#ATENÇÃO#

Considerações Finais

Nesse estudo abordamos os cenários mais comuns de análise e projetos orientados a objeto. Estudamos também como utilizar as técnicas de maneira prática via um processo que possui a essência de vários outros como o RUP. Apresentamos também alguns dos padrões de análise que geralmente não são tão conhecidos como os padrões de projeto. Com isso, esperamos aumentar sua caixa de ferramenta sobre o assunto e que isso possa melhorar a forma como você desenvolve software. Como uma última recomendação, utilize os padrões com sabedoria e sempre para melhorar a qualidade do software e não para adicionar mais complexidade sem a necessidade.

Referências

LARMAN, Craig. Utilizando UML e Padrões: Uma Introdução à Análise e ao Projeto Orientados a Objetos e ao Desenvolvimento Iterativo - 3ª Ed., Bookman, 2007.

FREEMAN, Elisabeth; FREEMAN, Eric. Use a Cabeça! Padrões de Projetos - 2ª Ed., Alta Books, 2007.

GAMMA, Erich et al. Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos. Bookman, 2000.

WAZLAWICK, Raul, Análise e Design Orientados a Objetos Para Sistemas de Informação, 2014.

ENGHOLM, Hélio, Análise e Design Orientados a Objetos, 2017

EVANS, Eric, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley Professional; Edição: 1, 2003

IIBA, Um Guia Para o Corpo de Conhecimento de Análise de Negócios - Guia Babok, International Institute of Business Analysis, 2015

PENDER, Tom, Uml a Bíblia, Elsevier, 2004

FOWLER, Martin. Analysis Patterns: Reusable Object Models, Addison-Wesley Professional, 1996.